NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 21. August 2026, Unterphase 4.2 abgeschlossen und validiert

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858; Auftragsbasis 46d73fa; Sionna und Sionna RT 2.0.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive
Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also
Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von
Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle
Stand umfasst die Unterphasen 1.1 bis 1.4, die vollständige Phase 2 sowie
die Unterphasen 3.1 bis 3.5 sowie 4.1: den numerischen SU-MIMO-Kern, konfigurierbare
Sionna-Arrays, einen
dimensionssicheren MIMO-Datenvertrag, explizit kalibrierbare
802.11ax-EESM-/PER-Abbildung und die
schmale Einkopplung in den SpectrumWifiPhy-Empfangspfad. SionnaMimoHelper legt
den High-Level-API-Vertrag und dessen zentrale Validierung fest und wendet ihn
nun konsistent auf Sionna-IPC, Phase-1-Linkabstraktion und SpectrumWifiPhy an.
Der MCS-bezogene ApplyForHeMcs-Aufruf prüft eine für SU-MIMO erforderliche
EESM-Kalibrierung vor jeder Zustandsänderung. Das reproduzierbare Beispiel
konfiguriert 1x1 und Vollrang-2x2 ausschließlich über SionnaMimoHelper und
deckt mit dem deterministischen Kanalpfad SpectrumWifiPhy end-to-end ab.
Integrationsnahe Negativläufe prüfen widersprüchliche NSS, nicht unterstützte
Modi und eine CFR-/PHY-Dimensionsverletzung. Ein dimensionssicherer,
benutzergegebener Precoder unterstützt nun frequenzflaches und
frequenzselektives festes W[k], einschließlich kontrollierter Streamauswahl
mit N_s < N_tx. Darauf aufbauend erzeugt ns-3 nun für den eindeutigen
MISO-Fall aus perfekter CFR ein leistungsnormalisiertes Ein-Stream-MRT-W[k].
Für N_tx = 1 ist der bestehende Ein-Stream-MMSE-Pfad als äquivalente
MRC-Kombination validiert. Die neue deterministische SVD-Fabrik wählt pro
Subcarrier explizit die N_s stärksten numerisch tragfähigen Eigenmoden und
erzeugt bei gleicher Leistung oder optionaler räumlicher Wasserfüllung erneut
ein festes W[k]. Rang, Singularwerte, Rauschleistung, Referenz-SNR, aktive
Moden und Leistungsallokation bleiben vollständig sichtbar. Die neue
semantische HE-/EHT-SU-Feedbackdarstellung komprimiert gleichleistungs-
SVD-Precoder getrennt in Givens-Winkel, quantisiert sie deterministisch mit
den groben 4/2- oder feinen 6/4-Bit-Codebooks und rekonstruiert daraus erneut
ein auf den festen-W[k]-Vertrag normalisiertes Ergebnis. Bitumfang,
Normierungskorrektur sowie phaseninvariante Richtungs- und Koeffizientenfehler
werden ausgewiesen. MimoCsiAgingModel trennt H_hat[k,t_csi] von H[k,t_tx],
bildet periodische Aktualisierung und Feedbackverzögerung explizit ab und
liefert eine gerichtete, array-, modus- und zeitstempelabhängige
Zustandsidentität. SionnaMimoHelper integriert nun festes Precoding, MRT, SVD,
Wasserfüllung, quantisiertes Feedback und CSI-Alterung durch eine explizite,
unveränderliche Precoderwahl. Apply führt das ausgewählte W[k] ohne geänderte
Semantik durch denselben festen SpectrumWifiPhy-Adapterpfad. Das neue
CPU-End-to-End-Beispiel vergleicht Open Loop, festen 2x1-Precoder, MRT und SVD
bei gleicher Gesamtleistung und gibt Konfiguration, Durchsatz, PER,
Stream-SINR, Rang, Precodingnorm, Leistungsanteile, Feedbackdiagnostik und
CSI-Alter maschinenlesbar aus.
Der additive Mehrnutzer-Kanalvertrag ordnet nun gerichtete, zeitgleiche CFRs
stabil nach Nutzer-ID, trägt die RX-Geometrie je Nutzer und übernimmt erst
nach vollständiger Snapshotprüfung Einträge in den bestehenden Cache. Darauf
wertet der neue native Downlink-MU-Kern einen festen gemeinsamen Precoder mit
expliziter Spaltenzuordnung aus. Er erhält die gesamte AP-Sendeleistung,

trennt gewünschte Leistung und Inter-User-Interferenz je Nutzer, Subcarrier und Stream und berechnet EESM sowie PER mit den nutzerspezifischen MCS- und Paketgrößenangaben. Precoderwahl, ZF/RZF, Scheduler, High-Level-Helper- und HE-MU-PHY-Integration aus Unterphase 4.3 und später wurden nicht begonnen.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

- Ubuntu 24.04, GCC 13.3, CMake 3.28
- ns-3.40 mit Tests und Beispielen
- ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
- Python 3.12.12, Sionna 2.0.1, Sionna RT 2.0.1 und NumPy 2.5.2
- Mitsuba 3.8.0, Dr.Jit 1.3.1 und LLVM-Backend llvm_ad_mono_polarized
- PyTorch 2.9.1+cpu; TensorFlow ist keine Laufzeitabhängigkeit mehr

Ohne explizite Arraykonfiguration verwendet der Server weiterhin exakt die bisherige 1x1-Sende- und Empfangsantenne. Die normalisierte SISO-CFR erfüllt

$$
\frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Unterphase 1.2 verallgemeinert diese Trennung auf alle Antennenpaare: H[k] enthält Small-Scale-Fading und der skalare Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.
2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

- CFR-Tensor H[k] mit expliziten Dimensionen N_sc x N_rx x N_tx.
- Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.
- Gleiche Gesamtleistungsaufteilung über N_s = N_tx Streams.
- Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.
- Post-MMSE-SINR pro Subcarrier und Stream.
- Mutual-Information-basierte Kompression über Subcarrier.
- Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10 reproduziert.

UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3 (IMPLEMENTIERT)

- Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
- Flache Reihenfolge sc/rx/tx verbindlich festlegen.
- Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der Antennendimensionen serialisieren.
- CacheEntry auf MimoChannelMatrix umstellen.

  - Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte
kommen indexgenau in ns-3 an.

  UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION (IMPLEMENTIERT)

  - Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite,
  Rauschzahl und thermischem Rauschen berechnen.
  - Effektive SINR mit einer explizit kalibrierbaren EESM-Abbildung je MCS
  bestimmen; ohne belegte Kalibrierung existieren bewusst keine Default-Betas.
  - Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
  - SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im
SISO-Grenzfall deckungsgleich zur festgelegten Referenz. Alle Kriterien wurden
mit den in Abschnitt 7 beschriebenen automatischen Tests erfüllt.

  UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL (IMPLEMENTIERT)

  - Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
  - NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh
  ablehnen.
  - Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
  - Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert
auf Rang-1-Kanälen nachvollziehbar gegen 1x1.
Die automatisierten Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

  PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam
konfiguriert:

  - Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
  - Ausrichtung: Azimut, Elevation und Rollwinkel
  - Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
  - Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp
  - Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass
Beispiele keine internen Objekte manuell verdrahten müssen.

Phase 2 wird in drei Unterphasen nach API-Vertrag, interner Integration und
Benutzbarkeit getrennt. Damit bleiben API-Entscheidungen ohne Änderung des
PHY-Verhaltens prüfbar, während Kompatibilitäts- und End-to-End-Regressionen
den nachfolgenden Integrationsschritten eindeutig zugeordnet werden können.

  UNTERPHASE 2.1: API-VERTRAG (IMPLEMENTIERT)

  - SionnaMimoHelper sowie Konfigurationstypen, dokumentierte Defaults und
  unterstützte Betriebsarten definieren.
  - Arraygeometrie, knotenspezifische Ausrichtung, TX-/RX-Antennenzahlen,
  maximale NSS, Empfängertyp, Kanalnormalisierung, Rauschzahl und
  Effective-SINR-Modell in einem konsistenten Vertrag zusammenführen.
  - Zentrale Validierung für vollständige, endliche und untereinander
  konsistente Parameter bereitstellen.
  - In Phase 2 nur SISO und SU-MIMO Spatial Multiplexing als funktionale Modi
  unterstützen. Beamforming und MU-MIMO werden entweder noch nicht angeboten
  oder brechen, falls sie für API-Stabilität bereits repräsentiert werden,

  eindeutig als nicht unterstützt ab.
  - Reine API-Unit-Tests für Defaults sowie gültige und ungültige
  Konfigurationen hinzufügen, ohne das bestehende PHY-Verhalten zu ändern.

Abnahmekriterium: Der API-Vertrag ist unabhängig von einer Simulation
testbar, rückwärtskompatible Defaults sind festgeschrieben und kein Modus aus
Phase 3 oder 4 erscheint fälschlich funktionsfähig.

  UNTERPHASE 2.2: INTEGRATION (IMPLEMENTIERT)

  - SionnaMimoHelper mit den vorhandenen Phase-1-Komponenten für Kanal,
  Link Budget, MMSE, EESM und SpectrumWifiPhy verbinden.
  - Knotenbezogene Arraygeometrie und Ausrichtung über den Sionna-IPC- und
  Serverpfad anwenden.
  - Antennenzahlen, maximale Spatial Streams, Rauschzahl, Empfängertyp,
  Normalisierung und Effective-SINR-Konfiguration konsistent an die
  beteiligten ns-3-Objekte weiterreichen.
  - Den bisherigen SionnaHelper-Pfad und die globalen 1x1-Defaults unverändert
  nutzbar halten.
  - Integrationstests ergänzen, welche manuelle Phase-1-Konfiguration und
  Helper-Konfiguration für 1x1 und 2x2 vergleichen.

Abnahmekriterium: Äquivalente manuelle und High-Level-Konfigurationen liefern
dieselben Phase-1-Ergebnisse; bestehende 1x1-Beispiele und Tests bleiben
unverändert erfolgreich.

  UNTERPHASE 2.3: BENUTZBARKEIT UND VALIDIERUNG (IMPLEMENTIERT)

  - Minimale 1x1- und 2x2-Beispiele ausschließlich über die High-Level-API
  anlegen.
  - Integrationsnahe Negativtests für widersprüchliche Laufzeitparameter und
  Fehlerweitergabe vom Helper bis zum PHY ergänzen.
  - End-to-End-Tests vom Helper über Sionna beziehungsweise den
  deterministischen Kanalpfad bis SpectrumWifiPhy ausführen.
  - API, Defaults, unterstützte Modi, Fehlermeldungen und Migration von der
  manuellen Konfiguration dokumentieren.

Abnahmekriterium: Die 1x1-/2x2-Beispiele laufen ohne manuelle Verdrahtung
interner Objekte, ungültige Konfigurationen brechen früh und eindeutig ab und
Dokumentation sowie PDF beschreiben den abgeschlossenen Stand von Phase 2.
Die helperbasierten End-to-End- und Laufzeit-Negativtests in Abschnitt 7
erfüllen dieses Kriterium.

  PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Sionna liefert weiterhin ausschließlich die komplexe CFR H[k]. Auswahl und
Anwendung des Precoding W[k], Streamauswahl, Leistungsaufteilung und
CSI-Zeitbezug bleiben vollständig in ns-3. Die bestehende skalare Grenze zu
SpectrumWifiPhy bleibt erhalten: Erst nach der matrixwertigen Kanal-,
Precoding- und Detektorberechnung wird auf eine Effective-SINR beziehungsweise
äquivalente Empfangs-PSD komprimiert.

Phase 3 ist inhaltlich größer als Phase 1 oder Phase 2 jeweils für sich: Sie
erweitert gleichzeitig Datenvertrag, numerischen Kern, High-Level-API und
PHY-Integration und führt zusätzlich rückgemeldete sowie zeitabhängige CSI
ein. Die folgenden fünf Unterphasen beschreiben deshalb vertikale,
abnehmbare Funktionen; die einzelnen Aufzählungspunkte bleiben interne
Implementierungsschritte. Unterphasen 3.1 bis 3.3 bilden den idealen
Perfect-CSI-Kern, Unterphase 3.4 ist der eigene Realismus- und
Standardisierungsblock und Unterphase 3.5 schließt die Phase end-to-end ab.

  UNTERPHASE 3.1: FESTES PRECODING ALS VOLLSTÄNDIGE GRUNDLAGE (IMPLEMENTIERT)

  - Einen dimensionssicheren Vertrag für
  W[k] in C^(N_tx x N_s) mit verbindlicher sc/tx/stream-Reihenfolge
  definieren; frequenzflache Matrizen werden explizit auf alle Subcarrier
  abgebildet.
  - Das Signalmodell auf
  y[k] = sqrt(P/N_s) H[k] W[k] x[k] + n[k] verallgemeinern und den bestehenden
  MMSE-Kern auf den effektiven Kanal H_eff[k] = H[k]W[k] anwenden.
  - N_s unabhängig von N_tx im Bereich 1 <= N_s <= min(N_tx, N_rx) führen.
  - Die Leistungsnormierung ||W[k]||_F^2 = N_s festlegen, sodass P weiterhin die
  gesamte Sendeleistung bezeichnet und kein unbeabsichtigter Precoding-Gewinn
  entsteht.
  - Analytische Tests für SISO, Identitäts-Precoding, Streamauswahl,
  komplexwertige feste Matrizen und Dimensions-/Normierungsfehler ergänzen.

Numerisches Zwischenkriterium: W[k] = I reproduziert den Phase-1-Kern
innerhalb 1e-10;
synthetische H[k]W[k]-Referenzen liefern indexgenau dieselben
Post-MMSE-SINRs und die Gesamtleistung bleibt invariant.

Integrationsschritte innerhalb derselben Unterphase:

  - Benutzergegebenes frequenzflaches und frequenzselektives W[k] durch
  SionnaMimoHelper, Link-Abstraktion und SpectrumWifiPhy-Adapter führen.
  - WifiTxVector::GetNss an die Spaltenzahl von W[k] statt weiterhin zwingend
  an N_tx koppeln und N_s < N_tx erstmals kontrolliert zulassen.
  - Subcarrierzahl, Antennenmaße, NSS, endliche Koeffizienten und
  Leistungsnormierung vor jeder Zustandsänderung zentral validieren.
  - Ohne explizites Precoding den bisherigen SISO-/Open-Loop-Pfad byte- und
  funktional kompatibel halten.
  - API-, Adapter- und End-to-End-Tests für 1x1, 2x2-Identität sowie
  N_tx = 2, N_rx = 2, N_s = 1 ergänzen.

Abnahmekriterium: Manuell und über SionnaMimoHelper konfiguriertes festes
W[k] liefern dieselben SINR-, EESM- und PER-Ergebnisse; alle Phase-2-Tests
bleiben unverändert erfolgreich. Die analytischen, Adapter- und
SpectrumWifiPhy-End-to-End-Ergebnisse in Abschnitt 7 erfüllen dieses
Kriterium.

  UNTERPHASE 3.2: EIN-STREAM-MRT/MRC (IMPLEMENTIERT)

  - MRT für den eindeutig definierten MISO-Fall N_rx = 1 aus perfekter CFR
  berechnen und auf Einheitsnorm normieren.
  - Für N_tx = 1 die bestehende Ein-Stream-MMSE-Lösung als äquivalente
  MRC-Kombination ausweisen und validieren.
  - Den allgemeinen Fall N_tx > 1 und N_rx > 1 nicht mehrdeutig als MRT/MRC
  bezeichnen; dessen optimale Ein-Stream-Lösung gehört zur SVD-Unterphase.
  - Analytische MISO-/SIMO-Kanäle, gemeinsame Phasendrehung, Nullkanal und
  erwarteten Arraygewinn testen.

Abnahmekriterium: MRT/MRC erreicht auf analytischen Ein-Stream-Kanälen die
Post-Combining-SNR rho ||h||_2^2, bleibt leistungsnormalisiert und fällt im
1x1-Grenzfall exakt auf das bestehende Ergebnis zurück. Der analytische
MRT-/MRC-Test in Abschnitt 7 erfüllt dieses Kriterium einschließlich
Phaseninvarianz und eindeutiger Ablehnung der Null- und allgemeinen
MIMO-Fälle.

  UNTERPHASE 3.3: SVD-PRECODING UND LEISTUNGSALLOKATION (IMPLEMENTIERT)

  - Pro Subcarrier H[k] = U[k] Sigma[k] V^H[k] zerlegen und die N_s stärksten

rechten Singulärvektoren bei zunächst gleicher Streamleistung als W[k] verwenden.
- N_s explizit wählen und gegen Antennenzahlen sowie einen dokumentierten numerischen Rangschwellwert validieren.
- Sortierung, Vorzeichen-/Phasenkonvention und Verhalten bei gleichen Singularwerten deterministisch festlegen.
- Diagonale, unitär gedrehte, frequenzselektive und rangdefiziente Referenzkanäle gegen analytische Eigenmode-SINRs testen.

Zwischenkriterium für SVD und Streamauswahl: Die Eigenmode-SINRs stimmen innerhalb 1e-10 mit
den Singularwertreferenzen überein; Rangverlust wird durch eine explizite Streamwahl behandelt und nicht durch versteckte Regularisierung kaschiert.

Optionale Leistungsallokation innerhalb derselben Unterphase:

- Gleiche Leistung als reproduzierbaren Standard beibehalten und Wasserfüllung als explizit auswählbare Forschungsoption ergänzen.
- Den Umfang zunächst auf räumliche SVD-Moden je Subcarrier begrenzen, damit spektrale Maske und Gesamtleistung pro Subcarrier unverändert bleiben.
- Rauschleistung, Referenz-SNR, aktive Moden und die Summe der zugewiesenen Leistung vollständig ausgeben und validieren.
- Analytische Zwei-Moden-Fälle unterhalb und oberhalb der Aktivierungsschwelle sowie Leistungserhaltung und KKT-Bedingungen testen.

Abnahmekriterium der optionalen Erweiterung: Die Leistungsanteile sind endlich, nicht negativ und
summieren sich exakt zur vorgegebenen Gesamtleistung; Wasserfüllung erreicht auf den Referenzkanälen mindestens die ideale Rate der geprüften Gleichleistungsreferenz, ohne Sendeleistung zu erzeugen.

UNTERPHASE 3.4: QUANTISIERTE RÜCKMELDUNG UND VERALTETE CSI (IMPLEMENTIERT)

- Vor der Implementierung das konkret unterstützte HE-/EHT-Feedbackformat, Winkel-/Codebookdarstellung, Bitbreiten und Rekonstruktionsreihenfolge verbindlich dokumentieren; nicht unterstützte Standardvarianten klar abgrenzen.
- Ideales SVD-Precoding, Quantisierung und Rekonstruktion als getrennte, einzeln testbare Schritte modellieren.
- Quantisiertes W[k] erneut leistungsnormalisieren und Feedbackumfang sowie Quantisierungsfehler als Diagnosegrößen bereitstellen.
- Bekannte Vektoren, Grenzwinkel, minimale/maximale Quantisierung und den unquantisierten Grenzfall testen.

Zwischenkriterium für die Quantisierung: Bitgenaue Referenzvektoren werden deterministisch
rekonstruiert, die Precoder bleiben leistungsnormalisiert und feinere Quantisierung verschlechtert die Referenzmetrik auf den Testkanälen nicht.

CSI-Alterung innerhalb derselben Unterphase:

- Die zur Precoderwahl verwendete Schätzung H_hat[k,t_csi] strikt von der bei der Paketübertragung wirksamen CFR H[k,t_tx] trennen.
- CSI-Alter, Aktualisierungsintervall und Feedbackverzögerung explizit und reproduzierbar konfigurieren; perfektes aktuelles CSI bleibt der Nullalter-Grenzfall.
- Richtung, Arraykonfiguration, Precodingmodus und CSI-Zeitstempel in Cache- beziehungsweise Zustandsidentität berücksichtigen.
- Statische Kanäle, kontrollierte Phasen-/Subraumrotationen und wiederholte Läufe mit identischen Seeds testen.

Abnahmekriterium: CSI-Alter null reproduziert das ideale Precoding exakt; kontrolliert veraltete Schätzungen werden nachweislich nur zur Wahl von W verwendet, während SINR und PER stets mit dem aktuellen H berechnet werden. Die bitgenauen Quantisierungs-, Leistungs-, Timing-, Subraum-, SINR- und PER-Regressionen in Abschnitt 7 erfüllen dieses Kriterium für den in Abschnitt 5.12 abgegrenzten SU-Feedbackumfang.

UNTERPHASE 3.5: BENUTZBARKEIT UND PHASENVALIDIERUNG (IMPLEMENTIERT)

- SionnaMimoHelper erst nach erfolgreicher Implementierung die jeweiligen Beamforming-/Precodingmodi als unterstützt melden lassen.
- Das Beispiel ns3_wifi_sumimo_precoding.cc anlegen und Open-Loop, festes Precoding, MRT sowie SVD bei gleicher Gesamtleistung vergleichen; Wasserfüllung, Quantisierung und CSI-Alter werden als explizite Optionen zugeschaltet.
- Durchsatz, PER, Stream-SINR, Kanalrang, Precodingnorm, Leistungsanteile und CSI-Alter maschinenlesbar ausgeben.
- Negative Konfigurationstests und vollständige CPU-End-to-End-Regressionen ergänzen sowie API-Migration und Modellgrenzen dokumentieren.

Abnahmekriterium: Die Beispiele benötigen keine manuelle Verdrahtung interner Objekte, analytisch überlegene Precoder zeigen den erwarteten Gewinn ohne zusätzliche Sendeleistung, ungültige Konfigurationen brechen früh ab und alle Phase-1-/2-Regressionen bleiben erfolgreich.

Dieses Kriterium ist erfüllt: Der Helper meldet SU_BEAMFORMING erst mit dieser Integration als unterstützt, das neue Vergleichsbeispiel verwendet nur die öffentliche High-Level-Konfiguration und SelectPrecoding, alle vier Modi halten dieselbe Gesamtleistungsdefinition ein, ungültige Kombinationen brechen vor Zustandsänderungen ab und die CPU-Regressionen in Abschnitt 7 decken die alten sowie die neuen Pfade ab.

PHASE 4: MU-MIMO

Sionna liefert auch in Phase 4 ausschließlich gerichtete komplexe CFRs $H_u[k]$ für die einzelnen Verbindungen. Gemeinsame Nutzerwahl, Precoding, Leistungsaufteilung, Inter-User-Interferenz, Scheduling, Empfängermodell und PER-Abbildung bleiben vollständig in ns-3. Eine MU-Übertragung darf nur Kanäle aus einem konsistenten gemeinsamen Zeitschnappschuss verwenden.

Phase 4 ist größer als die vorherigen Einzelphasen. Insbesondere sind Downlink und Uplink nicht symmetrisch: Im Downlink sendet ein AP mit einem gemeinsamen Precoder, während im Uplink mehrere unabhängige Sender am AP gemeinsam detektiert werden. Die Phase wird daher in acht vertikale Unterphasen geteilt. Die Unterphasen 4.1 bis 4.6 liefern zuerst einen vollständig nutzbaren Downlink-MU-MIMO-Pfad; 4.7 und 4.8 ergänzen danach den Uplink, ohne den Abschluss des Downlink-Pfads zu blockieren.

UNTERPHASE 4.1: MEHRNUTZER-KANALSCHNAPPSCHUSS UND IDENTITÄT (IMPLEMENTIERT)

- Einen dimensionssicheren Vertrag für gerichtete CFRs $\{H_u[k]\}_{u=1}^{U}$ mit stabiler Nutzer-ID, Sender, Empfänger, Richtung und gemeinsamem CSI-Zeitstempel definieren.
- Subcarrierzahl, Frequenzraster, Normalisierung und die Trennung zwischen CFR und skalarem Wideband-Pfadverlust über alle Nutzer konsistent halten.
- Unterschiedliche Empfangsarraygeometrien und Antennenzahlen je Nutzer repräsentieren; die bisherige gemeinsame RX-Geometrie darf nicht stillschweigend auf alle Stationen übertragen werden.
- Vorhandene gebündelte Sionna-Anfragen und Cachepfade soweit möglich wiederverwenden und gemischte Zeitstempel, doppelte Nutzer, inkonsistente Dimensionen oder Richtungen vor jeder Zustandsänderung ablehnen.

- Python- und C++-Vertragstests für Nutzerreihenfolge, Serialisierung, Tensorindizes, heterogene RX-Dimensionen und Zeitidentität ergänzen.

Abnahmekriterium: Synthetische Mehrnutzer-CFRs kommen unabhängig von ihrer Eingabereihenfolge nutzer- und indexgenau in ns-3 an; jeder inkonsistente Schnappschuss wird früh abgelehnt und alle bisherigen SU-Verträge bleiben unverändert erfolgreich.

Dieses Kriterium ist erfüllt: Die vorhandene `ChannelState`-/`RxNodeInfo`-Bündelung trägt eine additive `UserChannelIdentity`; Python und C++ prüfen den gesamten Schnappschuss vor Verwendung, C++ sortiert nach stabiler Nutzer-ID und der Cache übernimmt erst danach alle Links gemeinsam. Die neuen synthetischen Verträge decken heterogene RX-Arrays ab. Sämtliche Phase-1- bis Phase-3-SU-Regressionen bleiben bestanden; Unterphase 4.2 wurde nicht vorweggenommen.

UNTERPHASE 4.2: FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL (IMPLEMENTIERT)

- Einen festen gemeinsamen Precoder

$$
W[k] = \left[W_1[k],\ldots,W_U[k]\right]
$$

mit expliziter Zuordnung seiner Spalten zu Nutzer und Stream definieren.
- Die Phase-3-Leistungskonvention auf die Gesamtzahl $N_{s,\mathrm{ges}}$ aller gleichzeitig gesendeten Streams erweitern: $\lVert W[k]\rVert_F^2=N_{s,\mathrm{ges}}$, sodass P weiterhin die gesamte AP-Sendeleistung bezeichnet. Jede abweichende Nutzer- oder Streamgewichtung muss explizit sein und in Summe dieselbe Gesamtleistung erhalten.
- Für jeden Nutzer gewünschten Anteil und Inter-User-Interferenz getrennt berechnen:

$$
y_u[k] = H_u[k]W_u[k]x_u[k]
+ \sum_{v\ne u} H_u[k]W_v[k]x_v[k] + n_u[k].
$$

- Den bestehenden linearen Empfänger mit der vollständigen Interferenzkovarianz auswerten und gewünschte Leistung, Interferenzleistung sowie Post-Processing-SINR je Nutzer, Subcarrier und Stream ausgeben.
- EESM und PER anschließend separat mit dem MCS und der Paketgröße des jeweiligen Nutzers bestimmen.

Abnahmekriterium: Ein einzelner Nutzer reproduziert Phase 3 innerhalb 10^{-10}; orthogonale Nutzer erzeugen keine Inter-User-Interferenz, korrelierte Referenzkanäle liefern die analytische Interferenz und die Gesamtsendeleistung bleibt invariant.

Dieses Kriterium ist erfüllt: `DlMuMimoPrecodingMatrix` ordnet jede Spalte des unveränderten Phase-3-Precoders explizit einem Nutzer und lokalen Stream zu. `DlMuMimoLinkModel` wertet die vollständige Empfangskovarianz aus und weist gewünschte Leistung, Inter-User-Interferenz und SINR getrennt aus. `HeDlMuMimoLinkAbstraction` wendet den bestehenden EESM- und ErrorRateModel-Vertrag anschließend nutzerweise an. Analytische native Tests erfüllen alle vier Invarianten; Unterphase 4.3 wurde nicht vorweggenommen.

UNTERPHASE 4.3: IDEALES DOWNLINK-ZF UND REGULARISIERTES ZF

- Zunächst den eindeutigen Minimalfall mit einem Stream je Nutzer und perfekter gleichzeitiger CSI implementieren.
- ZF aus dem gestapelten Mehrnutzerkanal und RZF mit explizit konfigurierbarem Regularisierungsparameter berechnen; es gibt keinen versteckten Regularisierungsdefault.
- Rangschwelle, Nutzerreihenfolge, Pseudoinverse und deterministische Behandlung gleicher beziehungsweise nahezu gleicher Singulärwerte dokumentieren.
- Den gemeinsamen Precoder nach jeder Auswahl auf die Konvention aus Unterphase 4.2 normalisieren und überlastete oder rangdefiziente Nutzergruppen eindeutig ablehnen beziehungsweise als nicht planbar melden.
- Orthogonale, korrelierte, überlastete und rangdefiziente analytische Referenzkanäle testen.

Abnahmekriterium: ZF unterdrückt auf vollrangigen Referenzkanälen die Inter-User-Interferenz innerhalb der numerischen Toleranz; RZF bleibt auf schlecht konditionierten Kanälen endlich, deterministisch und leistungsnormalisiert. Mehrere Streams je Nutzer werden erst nach einem explizit dokumentierten Blockdiagonalisierungs- oder gemeinsamen Transceiververtrag freigeschaltet und bis dahin klar abgelehnt.

UNTERPHASE 4.4: HE-DOWNLINK-MU-PHY-INTEGRATION

- Den gemeinsamen MU-Zustand in den schmalsten geeigneten SpectrumWifiPhy-Pfad für `WIFI_PREAMBLE_HE_MU` einkoppeln.
- Die vorhandenen STA-ID-, RU-, MCS- und NSS-Angaben aus `WifiTxVector::HeMuUserInfoMap` verwenden, anstatt parallele Wi-Fi-Metadaten im Sionna-Modul einzuführen.
- Inter-User-Interferenz nur zwischen räumlich gemultiplexten Nutzern innerhalb derselben RU berücksichtigen; verschiedene RUs bleiben orthogonal.
- Für jeden Empfänger erst nach der matrixwertigen MU-Berechnung auf dessen skalare äquivalente Empfangs-PSD komprimieren. Pfadverlust, AP-Sendeleistung, Rauschen und Empfangsgewinn dürfen dabei jeweils genau einmal eingehen.
- Positive und negative Adaptertests für Nutzer-/STA-ID-Zuordnung, RU, NSS, Antennendimensionen und unbekannte Empfänger ergänzen.

Abnahmekriterium: Ein echter HE-DL-MU-PPDU liefert je adressiertem Empfänger die analytisch erwartete SINR und PER; nicht adressierte oder inkonsistente Empfänger verändern keinen MU-Zustand. Der bestehende SU-Pfad bleibt bit- und funktionskompatibel. EHT bleibt bis zu einer expliziten EHT-Link-/PER-Abstraktion als nicht unterstützt gekennzeichnet.

UNTERPHASE 4.5: LAUFZEIT-CSI UND SCHEDULER-SCHNITTSTELLE

- Einen gerichteten, zeitgestempelten CSI- und Precoderzustand pro AP, Nutzergruppe, RU und Übertragungsrichtung verwalten; veraltete oder für eine andere Gruppe erzeugte Zustände dürfen nicht wiederverwendet werden.
- An die vorhandene ns-3-`MultiUserScheduler`-Schnittstelle anbinden und zunächst deterministische Nutzergruppen sowie RU-/NSS-Zuordnungen unterstützen.
- Danach eine explizit auswählbare korrelationsbasierte Gruppierung ergänzen, welche unbelegbare Gruppen verwirft und ihre Auswahlmetrik ausgibt.
- CSI-Aktualisierungsintervall, Feedbackverzögerung und den tatsächlichen Sendezeitpunkt aus Phase 3 übernehmen. Quantisierte MU-Rückmeldung erhält einen eigenen Vertrag; ein rekonstruierter SU-Precoder darf nicht stillschweigend als vollständige MU-Kanalschätzung behandelt werden.
- Wiederholte Läufe mit identischen Seeds sowie kontrollierte Änderungen von Nutzerreihenfolge, Korrelation und CSI-Alter testen.

Abnahmekriterium: Scheduler und PHY verwenden nachweislich denselben

gerichteten Kanalschnappschuss; unpassende oder veraltete Zustände werden erkannt und identische Eingaben erzeugen exakt dieselbe Nutzergruppe, RU-/NSS-Zuordnung und denselben Precoder.

  UNTERPHASE 4.6: DOWNLINK-BENUTZBARKEIT UND PHASENVALIDIERUNG

  - `SionnaMimoHelper` den Modus `MU_MIMO` erst nach erfolgreicher End-to-End-Integration als unterstützt melden lassen.
  - Das Beispiel `ns3_wifi_mumimo.cc` anlegen und SU-Round-Robin, festes MU-Precoding, MU-ZF und MU-RZF bei gleicher Gesamtleistung und vergleichbarer Sendezeit gegenüberstellen.
  - Orthogonale und stark korrelierte Nutzergruppen sowie aktuelles und veraltetes CSI vergleichen.
  - Pro Nutzer Durchsatz, PER, Stream-SINR, gewünschte Leistung, Inter-User-Interferenz, RU, NSS und CSI-Alter sowie für die Gruppe Rang, Precodingnorm, Gesamtleistung und Fairness maschinenlesbar ausgeben.
  - Vollständige CPU-End-to-End- und Negativtests sowie alle Regressionen der Phasen 1 bis 3 ausführen.

Abnahmekriterium: MU-ZF erzielt bei ausreichend unkorrelierten Nutzern den erwarteten gleichzeitigen Übertragungsgewinn gegenüber SU-Round-Robin, ohne zusätzliche AP-Sendeleistung zu erzeugen; bei hoher Korrelation verschlechtert sich das Ergebnis nachvollziehbar oder die Gruppe wird abgelehnt. Beispiele benötigen keine manuelle Verdrahtung interner Objekte.

  UNTERPHASE 4.7: UPLINK-MU-NUMERIKKERN

  - Das Uplink-Signalmodell

  \[
    y[k] = \sum_{u=1}^{U} H_u[k]x_u[k] + n[k]
  \]

  mit getrennten Nutzer-Sendeleistungen, Pfadverlusten und Streamzuordnungen definieren.
  - Zunächst ideale Zeit- und Frequenzsynchronisation sowie einen Stream je Nutzer annehmen und diese Grenze ausdrücklich dokumentieren.
  - Gemeinsame ZF- und lineare MMSE-Detektion am AP implementieren und Post-Processing-SINR, Inter-User-Interferenz und PER je Nutzer bestimmen.
  - Nah-Fern-Fälle, orthogonale Nutzer, Rangverlust, unterschiedliche Sendeleistungen und den Einzelnutzer-Grenzfall analytisch testen.

Abnahmekriterium: Der Einzelnutzerfall reproduziert die vorhandene SU-Empfangsabstraktion; analytische Mehrnutzerkanäle und Nah-Fern-Fälle liefern die erwarteten ZF-/MMSE-SINRs, ohne Nutzerleistungen zu vermischen.

  UNTERPHASE 4.8: UPLINK-PHY-INTEGRATION UND GESAMTABSCHLUSS

  - Den Uplink-Kern an koordinierte HE-TB-/UL-MU-Übertragungen, Triggerrahmen, STA-ID-, RU- und MCS-Zuordnungen anbinden.
  - Sicherstellen, dass nur tatsächlich gleichzeitig empfangene Nutzer in denselben Detektionszustand eingehen; fehlende, verspätete oder inkonsistent konfigurierte Teilnehmer werden explizit behandelt.
  - Das Phasenbeispiel um UL-MU gegenüber sequenziellen SU-Übertragungen erweitern und die ideale Synchronisationsannahme in der Ausgabe sichtbar machen.
  - Alle API-, Numerik-, PHY-, Scheduler-, CPU-End-to-End- und Legacytests ausführen sowie Dokumentation und PDF abschließend aktualisieren.

Abnahmekriterium: Eine koordinierte UL-MU-Gruppe liefert pro Nutzer reproduzierbare SINR-, PER- und Durchsatzwerte; der erwartete

Gleichzeitigkeitsgewinn sowie Nah-Fern- und Rangverlusteffekte sind sichtbar,
und alle Phase-1- bis Phase-3-Regressionen bleiben erfolgreich.

  PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

  - PER-Kurven gegen Link-Level-Referenzen kalibrieren.
  - Sensitivität gegenüber Antennenabstand, Orientierung, CSI-Alter und
  Raytracing-Samples untersuchen.
  - Cache- und IPC-Speicherbedarf für große N_sc, N_tx und N_rx messen.
  - Deterministische Seeds, Ergebnisformate und CI-Matrix für CPU/GPU festlegen.

4. DATENMODELL UND IPC-VERTRAG DER UNTERPHASEN 1.1/1.2

  4.1 TENSOR UND FLACHE REIHENFOLGE

Für jeden Subcarrier k ist

$$
  \mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container und die beiden Protocol-Buffer-Felder csi_real und csi_imag
speichern die Werte flach. Der Index für den Eintrag h[k,r,t] lautet

$$
  i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Es gelten die
verbindlichen Längen

$$
  |\mathrm{frequencies}| = N_\mathrm{sc},
  \qquad
  |\mathrm{csi_real}| = |\mathrm{csi_imag}|
  = N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}.
$$

Die neuen skalaren Felder num_rx_antennas und num_tx_antennas wurden mit den
Feldnummern 9 und 10 an RxNodeInfo angehängt. Der Proto3-Standardwert null wird
beim Empfänger als eins interpretiert. Für eine neue 1x1-Antwort lässt der
Python-Server beide Felder bewusst ungesetzt; ihre serialisierte Darstellung
ist dadurch bytegleich zur alten Nachricht. Alte Sender werden als 1x1 gelesen,
alte Empfänger ignorieren die unbekannten neuen Felder einer MIMO-Antwort.

  4.2 TRANSFORMATION AUS SIONNA

Sionna RT 2.0.1 liefert die CFR weiterhin in der Form

$$
  H_\mathrm{raw}
  [\ell,r,q,t,n,k]
$$

mit Linkindex ℓ, RX-Antenne r, TX-Index q, TX-Antenne t, Zeitindex n und
Subcarrier k. Eine ns3sionna-Anfrage enthält genau einen Sender und einen
Zeitpunkt. Deshalb wird ohne squeeze-Operation explizit ausgewählt:

$$
  H_\ell[r,t,k] =
  H_\mathrm{raw}[\ell,r,0,t,0,k].
$$

\]

Vor der Serialisierung wird nach [k,r,t] transponiert und anschließend in
C-Reihenfolge abgeflacht. Dadurch bleiben auch Singleton-Dimensionen sichtbar
und 2x2-Werte landen indexgenau im C++-Container.

4.3 WIDEBAND-VERLUST UND MIMO-NORMALISIERUNG

Für die rohe Link-CFR wird die mittlere Leistung gemeinsam über Subcarrier und
Antennenpaare gebildet:

\[
  \bar P =
  \frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
  \sum_{k=0}^{N_\mathrm{sc}-1}
  \sum_{r=0}^{N_\mathrm{rx}-1}
  \sum_{t=0}^{N_\mathrm{tx}-1}
  |H_\ell[r,t,k]|^2.
\]

Der übertragene skalare Wideband-Verlust und die normalisierte Matrix lauten

\[
  L_\mathrm{wb} = -10\log_{10}(\bar P)\ \mathrm{dB},
  \qquad
  H_\mathrm{norm}[r,t,k] =
  \frac{H_\ell[r,t,k]}{\sqrt{\bar P}}.
\]

Damit gilt

\[
  \frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
  \sum_{k,r,t}|H_\mathrm{norm}[r,t,k]|^2 = 1.
\]

Für N_rx=N_tx=1 ist dies exakt die bisherige SISO-Normalisierung. Die Aufteilung
der gesamten Sendeleistung auf N_s Streams bleibt davon getrennt und erfolgt
weiterhin im numerischen Modell mit dem Faktor 1/N_s.

4.4 ARRAYKONFIGURATION UND NS-3-CACHE

SimInitMessage besitzt optionale tx_array- und rx_array-Nachrichten mit Zeilen,
Spalten, vertikalem und horizontalem Abstand in Wellenlängen, Elementmuster und
Polarisation. Ohne Nachricht werden exakt die alten Arrays verwendet:
1x1/tr38901/V am Sender und 1x1/dipole/V am Empfänger. SionnaHelper bietet dafür
die kleinen Methoden ConfigureTransmitArray und ConfigureReceiveArray; die
umfassende knotenbezogene High-Level-API bleibt Phase 2 vorbehalten.

Die Antwortdimensionen werden aus der tatsächlich von Sionna gelieferten Form
abgeleitet und berücksichtigen damit auch die Polarisation. CacheEntry hält die
CFR nun als MimoChannelMatrix. GetPropagationChannelMatrix erhält die
Dimensionen; GetPropagationCSI bleibt als flache, für 1x1 unveränderte
Kompatibilitätsansicht bestehen.

4.5 MEHRNUTZER-SNAPSHOT UND GERICHTETE IDENTITÄT DER UNTERPHASE 4.1

Die bestehende gebündelte Antwort bleibt der Transportcontainer. Eine
`ChannelState`-Nachricht bezeichnet genau einen CSI-Zeitpunkt und enthält
weiterhin einen gemeinsamen Sender sowie mehrere `RxNodeInfo`-Nachrichten.
Feld 11 von `RxNodeInfo` ergänzt additiv die Nachricht

`UserChannelIdentity` mit

$$
  I_u=(u, n_{\mathrm{tx}}, n_{\mathrm{rx}}, d, t_{\mathrm{csi}}, A_{\mathrm{rx},u}).
$$

Hier sind u die stabile Nutzer-ID, n_{tx} und
n_{rx} die gerichteten Endpunkte, d die explizite Richtung
und $A_{\mathrm{rx},u}$ die vollständige Empfangsarraygeometrie. Unterphase
4.1 erzeugt ausschließlich `CHANNEL_DIRECTION_DOWNLINK`; Uplink wird damit
nicht vorweggenommen. Fehlt `UserChannelIdentity`, bleibt die vorhandene
SU-/Legacy-Deserialisierung unverändert.

Für einen gültigen Snapshot gelten gleichzeitig

$$
  t_{\mathrm{csi},u}=t_{\mathrm{start}},\qquad
  f_{u,k}=f_k,\qquad
  N_{\mathrm{sc},u}=N_{\mathrm{sc}},\qquad
  N_{\mathrm{tx},u}=N_{\mathrm{tx}}.
$$

Die Empfangsdimension darf dagegen nutzerabhängig sein. Für ein planares
Array mit R_u Zeilen, C_u Spalten und $P_u \in \{1,2\}$
Polarisationsports gilt verbindlich

$$
  N_{\mathrm{rx},u}=R_u C_u P_u.
$$

Jeder Nutzer trägt sein eigenes $A_{\mathrm{rx},u}$; die bisherige globale
RX-Konfiguration wird nicht mehr stillschweigend aus einem anderen Link
abgeleitet. Der aktuelle Server schreibt für jeden von ihm erzeugten
Mehrnutzer-Link die tatsächlich konfigurierte globale Geometrie explizit aus;
Einzelnutzerantworten bleiben identitätsfrei. Der IPC- und
C++-Vertrag kann bereits unterschiedliche Geometrien und
$N_{\mathrm{rx},u}$ innerhalb desselben synthetischen Snapshots abbilden;
eine knotenbezogene High-Level-Konfiguration des Sionna-Raytracings gehört
nicht zu dieser Unterphase.

Die CFR jedes Nutzers verwendet unverändert die Reihenfolge
$i_u(k,r,t) = (kN_{\mathrm{rx},u} + r)N_{\mathrm{tx}} + t,$ einen eigenen skalaren
Wideband-Pfadverlust und die Einheitsleistungsnormalisierung

$$
  \frac{1}{N_{\mathrm{sc}}N_{\mathrm{rx},u}N_{\mathrm{tx}}}
  \sum_{k,r,t}|H_u[k,r,t]|^2=1.
$$

`validate_multi_user_snapshot` prüft denselben Vertrag auf der Python-Seite.
`DeserializeMultiUserChannelSnapshot` prüft in C++ zunächst sämtliche Nutzer,
ohne Cachezustand zu verändern, und baut erst danach
`MultiUserChannelSnapshot` auf. Dessen Nutzervektor ist unabhängig von der
Eingabereihenfolge streng aufsteigend nach u sortiert. Gemischte oder
fehlende Identitäten, doppelte Nutzer, falsche Endpunkte/Richtungen,
Zeitstempel, Frequenzraster, TX-Dimensionen, nicht passende RX-Geometrien,
nicht endliche Werte und abweichende Normalisierung werden abgelehnt.

Auch Antworten mit mehreren Look-ahead-Zeitpunkten werden vollständig in
temporäre Einträge dekodiert, bevor der bestehende Cache einen davon
übernimmt. Explizit identifizierte Einträge dürfen nur in ihrer gespeicherten

TX-RX-Richtung getroffen werden. Identitätsfreie alte SU-Einträge behalten dagegen die bisherige reziproke Schlüsselverwendung. Damit bleibt der SU-Datenpfad erhalten, während Mehrnutzerzustände weder partiell noch in umgekehrter Richtung sichtbar werden.

5. NUMERISCHES SU- UND DOWNLINK-MU-MIMO-MODELL

5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
\mathbf{y}[k] =
\sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

mit $N_s = N_{tx}$, $E[x\,x^H] = I$ und räumlich weißem Rauschen $E[n\,n^H] = N\,I$. Die Eingangsgröße

$$
\rho = \frac{P}{N}
$$

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell verlangt in dieser Unterphase $N_{rx} >= N_{tx}$. Overloaded MIMO wird erst mit einer expliziten Streamauswahl zugelassen.

5.2 LINEARER MMSE-EMPFÄNGER

Mit $\alpha = \rho/N_s$ und $G[k] = H[k]^H H[k]$ folgt die Fehlerkovarianz des linearen MMSE-Detektors:

$$
\mathbf{E}[k] =
(\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
$$

Die Post-Processing-SINR des Streams s ist

$$
\gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
$$

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch definiert, weil $I + \alpha G$ positiv definit ist.

5.3 SISO-GRENZFALL

Für $N_{tx} = N_{rx} = 1$ ergibt sich

$$
E = \frac{1}{1 + \rho |h|^2},
\qquad
\gamma = \rho |h|^2.
$$

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem üblichen SISO-SNR-Modell.

5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine
Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

$$
  \gamma_{\mathrm{eff},s} =
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
$$

Die mittlere ideale spektrale Effizienz über alle Streams ist

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese MI-Größe bleibt eine Validierungsmetrik und ist keine unmittelbare
802.11-Datenrate oder PER. Die HE-PER-Abbildung verwendet stattdessen die
MCS-spezifische EESM-Kompression aus Abschnitt 5.6.

5.5 REFERENZ-SNR AUS DEM LINK BUDGET

Die skalare Wideband-Dämpfung L_wb bleibt von der normalisierten
Small-Scale-CFR getrennt. Aus der gesamten Sendeleistung $P_\mathrm{tx,dBm}$
folgen Empfangsleistung und lineare Empfangsleistung

$$
  P_\mathrm{rx,dBm} = P_\mathrm{tx,dBm} - L_\mathrm{wb},
  \qquad
  P_\mathrm{rx} =
  10^{(P_\mathrm{rx,dBm}-30)/10}\ \mathrm{W}.
$$

Mit Rauschbandbreite B in Hz, Temperatur T in Kelvin und
Empfängerrauschzahl NF in dB gelten

$$
  N_\mathrm{th} = kTB,
  \qquad
  F = 10^{NF/10},
  \qquad
  N_\mathrm{rx} = FkTB,
$$

und damit

$$
  \rho =
  \frac{P_\mathrm{rx}}{N_\mathrm{rx}}
  =
  \frac{10^{(P_\mathrm{tx,dBm}-L_\mathrm{wb}-30)/10}}
    {kTB\,10^{NF/10}}.
$$

WifiLinkBudget gibt Empfangsleistung, thermisches Rauschen,
Empfängerrauschen und ρ separat aus. dB-/Linear- und dBm-/Watt-
Umrechnungen delegieren an DbToRatio, RatioToDb und DbmToW aus
ns-3.40 wifi-utils. Für $k=1{,}3803\cdot10^{-23}\ \mathrm{J/K}$ wird bewusst

derselbe Zahlenwert wie in ns-3.40 InterferenceHelper verwendet. Das ist die Provenienz dieser physikalischen Konstante; es ist keine Fit-Konstante. Anders als InterferenceHelper ist T hier ein expliziter positiver Eingabeparameter und nicht fest auf 290 K gesetzt.

P_tx ist die Gesamtleistung über alle Spatial Streams; die Aufteilung durch N_s erfolgt erst im MMSE-Modell. B ist die Empfänger-Rauschbandbreite. Link Budget und MMSE addieren selbst keine externe Interferenz; der SpectrumWifiPhy-Pfad bewertet sie anschließend skalar gemäß Abschnitt 5.8. Ein zusätzlicher pauschaler RX-Diversity-Gewinn wird nicht aufgeschlagen: Array- und Detektorgewinn sind bereits in $\mathbf H[k]$ und den Post-MMSE-SINRs enthalten.

5.6 MCS-ABHÄNGIGE EESM-KOMPRESSION

Für HE MCS m komprimiert EESM alle $N_\mathrm{sc}N_s$ Post-MMSE-SINRs mit gleicher Gewichtung:

$$
\gamma_{\mathrm{EESM},m}
=
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k=0}^{N_\mathrm{sc}-1}
  \sum_{s=0}^{N_s-1}
  \exp\left(-\frac{\gamma_s[k]}{\beta_m}\right)
\right).
$$

$\beta_m>0$ hat wie γ lineare SINR-Einheiten und ist MCS-abhängig. SetEesmBeta erwartet den linearen Wert; SetEesmBetaDb ist die dB-Komfortfunktion. Zur Vermeidung numerischen Unterlaufs wird mit $\gamma_\min=\min_{k,s}\gamma_s[k]$ äquivalent gerechnet:

$$
\gamma_{\mathrm{EESM},m}
=
\gamma_\min
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k,s}
  \exp\left(-\frac{\gamma_s[k]-\gamma_\min}{\beta_m}\right)
\right).
$$

Für einen flachen Kanal mit identischen SINRs ist das Ergebnis unabhängig von β_m exakt diese SINR. Das ermöglicht die SISO-AWGN-Paritätsprüfung.

Es werden bewusst keine Default-Betas ausgeliefert. Jeder HE MCS von 0 bis 11 muss vor der Nutzung explizit konfiguriert werden. Damit enthält der Code keine unbelegten Kalibrierkonstanten. Die in analytischen Tests verwendeten positiven Werte sind ausschließlich Formel-Sentinels und ausdrücklich keine 802.11ax-Kalibrierung. Belastbare β_m müssen später aus dokumentierten Link-Level-Kurven gefittet und zusammen mit Datensatz, PHY-Konfiguration, Codeart und Paketlänge versioniert werden. Die bisherige MI-Kompression je Stream bleibt als modellunabhängige Diagnosemetrik erhalten und wird nicht als kalibrierte PER-Abbildung ausgegeben.

5.7 ABBILDUNG AUF NS-3-PER

HeSuMimoLinkAbstraction übernimmt WifiTxVector einschließlich HE-MCS, NSS und

Codeart. NSS muss der Zahl der übertragenen Streams entsprechen. Nach EESM wird die vorhandene Schnittstelle unverändert verwendet:

$$
  P_\mathrm{succ} =
  \mathrm{ErrorRateModel::GetChunkSuccessRate}
  (m,\mathrm{TxVector},\gamma_{\mathrm{EESM},m},8L,N_\mathrm{rx}),
  \qquad
  PER = 1-P_\mathrm{succ},
$$

wobei L die Paketlänge in Byte ist. ErrorRateModel ist als ns-3-Attribut und über SetErrorRateModel konfigurierbar. Standard ist TableBasedErrorRateModel, also das ns-3.40-Defaultmodell für 802.11n/ac/ax.

Die Kalibrierprovenienz bleibt vollständig bei ns-3: Laut src/wifi/doc/source/wifi-design.rst stammen die AWGN-PER-Tabellen aus End-to-End-Linksimulationen mit MATLAB WLAN Toolbox, wurden gegen IEEE-TGn-Ergebnisse validiert und verwenden die Paketlängenextrapolation aus IEEE P802.11 TGax 11-14/0803r1. BCC besitzt Referenztabellen für 32 und 1458 Byte mit einer 400-Byte-Schwelle und MCS 0 bis 9; MCS 10/11 fällt auf YansErrorRateModel zurück. LDPC besitzt eine 1458-Byte-Referenz für MCS 0 bis 11. Die neue Abstraktion kopiert oder verändert keine dieser Kurven und fügt keinen SNR-Offset hinzu. Da sie Erfolgswahrscheinlichkeiten und keine Zufallsentscheidung ausgibt, ist die Berechnung bei identischen Eingaben deterministisch.

5.8 EINKOPPLUNG IN SPECTRUMWIFIPHY

Die Integration bleibt auf SionnaSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity beschränkt. Für einen HE-SU-PPDU liest sie N_s ausschließlich mit WifiTxVector::GetNss. Vor der Link-Abstraktion müssen die physikalischen TX-/RX-Antennenzahlen gleich den CFR-Dimensionen, die maximal unterstützten Spatial Streams mindestens N_s und $N_\mathrm{ant,rx}\geq N_s$ sein. Ohne konfiguriertes Precoding bleibt zusätzlich unverändert $N_s=N_\mathrm{tx}$. Mit festem Precoding muss N_s dagegen exakt der Spaltenzahl von W[k] entsprechen. Inkonsistenzen brechen damit vor MMSE, EESM und Paketempfang ab. Der neue kontrollierte Fall $N_s<N_\mathrm{tx}$ ist ausschließlich mit explizitem festem W[k] zulässig.

Der SpectrumChannel hat den skalaren Wideband-Pfadverlust bereits auf params->psd angewendet. Die MIMO-Integration verwendet daher

$$
  P_\mathrm{in} =
  \int S_\mathrm{params}(f)\,\mathrm df \; G_\mathrm{rx}
$$

als bereits pfadverlustbehaftete Empfangsleistung. Der Wideband-Verlust aus dem Cache wird an dieser Stelle nur noch für den bestehenden Diagnose-Tag gelesen und nicht erneut multipliziert. Empfangsverstärkung, Rauschbandbreite, 290 K und Empfängerrauschzahl gehen genau einmal in das Link Budget ein. Weil ns-3.40 die Rauschzahl nicht lesbar als WifiPhy-Attribut bereitstellt, verlangt SetReceiverNoiseFigureDb einen expliziten Wert; er muss mit SpectrumWifiPhy::RxNoiseFigure identisch konfiguriert werden. Ein fehlender Wert bricht den MIMO-Pfad früh ab.

MMSE und EESM liefern γ_eff. SpectrumWifiPhy erwartet dagegen weiterhin eine skalare PSD und wendet seine InterferenceHelper-Berechnung später selbst an. Für das AWGN-Fehlermodell ist dessen interner

Diversity-Faktor

$$
  G_\mathrm{AWGN} =
  \begin{cases}
    N_\mathrm{ant,rx}/N_s, & N_\mathrm{ant,rx}>N_s,\\
    1, & \text{sonst}.
  \end{cases}
$$

Da der MMSE-Schritt den Arraygewinn bereits enthält, wird die äquivalente skalare Empfangsleistung auf

$$
  P_\mathrm{eq} =
  \frac{\gamma_\mathrm{eff} P_N}{G_\mathrm{AWGN}},
  \qquad P_N = kTB\,10^{NF/10}
$$

gesetzt. InterferenceHelper erhält damit bei zusätzlicher skalarer Interferenz I

$$
  \gamma_\mathrm{phy}
  = \frac{P_\mathrm{eq}}{P_N+I}G_\mathrm{AWGN}
  = \gamma_\mathrm{eff}\frac{P_N}{P_N+I}.
$$

Ohne Interferenz gilt exakt $\gamma_\mathrm{phy}=\gamma_\mathrm{eff}$; Pfadverlust, thermisches Rauschen, Rauschzahl und Arraygewinn werden nicht doppelt angewendet. Zeitvariable Interferenz wirkt weiterhin über SpectrumWifiPhy, jedoch als skalare Nachbewertung und nicht als subcarrierabhängige Interferenzkovarianz im MMSE-Detektor.

Der bisherige 1x1-Pfad bleibt unverändert skalar und benötigt kein EESM-Beta. Nicht-HE-Kontrollrahmen einer MIMO-PHY verwenden während Phase 1 das erste Antennenpaar, damit Beacon, Association und ACK ohne vorweggenommene Streamauswahl funktionieren. Der letzte Guard-Bin und das bestehende CFR-Tag-Format bleiben im 1x1-Fall kompatibel. Für automatisierte Beispiele kann eine normalisierte feste CFR den Sionna-Cache ersetzen; dieser deterministische Test-Hook ändert den Produktionspfad nicht. Der MimoRx-Trace meldet PPDU-UID, Knoten, Effective-SINR, PER und den kleinsten Kanalrang über alle Subcarrier.

Das Beispiel ns3_wifi_sumimo_minimal.cc erzeugt drei flache, normierte Kanäle bei HE MCS 5 und 20 dB Referenz-SNR: 1x1 mit $h=1$, gut konditioniertes 2x2 mit $\mathrm{diag}(\sqrt{2},\sqrt{2})$ und Rang-1-2x2 mit ausschließlich Einsen. Zusätzlich führt es den gut konditionierten Kanal mit explizitem Identitäts-Precoding und mit einem 2x1-Precoder zur Auswahl der ersten TX-Antenne aus. Es misst realen UDP-Durchsatz über SpectrumWifiPhy und gibt zusätzlich PER, Stream-SINR, EESM-SINR und Mindestkanalrang aus. Der Aufruf muss ein positives --eesmBetaLinear liefern. Der im Regressionstest verwendete Wert 2 ist nur ein expliziter Formel-Sentinel; bei diesen flachen Kanälen ist das Ergebnis betaunabhängig und stellt keine Produktionskalibrierung dar.

Die deterministische Referenzausgabe lautet:

- 1x1: Rang 1, Stream-/Effective-SINR 20,000 dB, PER 0,000016 und 68,1408 Mbit/s;
- gut konditioniertes 2x2: Rang 2, beide Stream-SINRs und Effective-SINR

20,000 dB, PER 0,000016 und 134,3904 Mbit/s;
- Rang-1-2x2: Rang 1, beide Stream-SINRs und Effective-SINR -0,043214 dB,
PER 1 und 0 Mbit/s;
- Identitäts-Precoding 2x2: dieselben analytischen SINR-, EESM- und
PER-Ergebnisse wie der offene Vollrangpfad und 133,4784 Mbit/s im
sequenziellen Netzwerkregressionslauf;
- 2x1-Streamauswahl auf dem 2x2-Kanal: Rang 1, Stream-/Effective-SINR
23,010300 dB, PER 0 und 68,1504 Mbit/s. Der 3,0103-dB-Unterschied zum
offenen Einzelstreamvergleich folgt aus der vollen Gesamtleistung für
$N_s=1$, nicht aus zusätzlicher Precoderleistung.

5.9 FESTES PRECODING DER UNTERPHASE 3.1

MimoPrecodingMatrix ist ein unveränderlicher, dimensionssicherer Werttyp für

$$
\mathbf W[k]\in\mathbb C^{N_\mathrm{tx}\times N_s},
\qquad
1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx}).
$$

Seine flache Reihenfolge ist verbindlich subcarrier/tx/stream:

$$
j(k,t,s)=(kN_\mathrm{tx}+t)N_s+s.
$$

Der frequenzselektive Konstruktor erwartet deshalb genau
$N_\mathrm{sc}N_\mathrm{tx}N_s$ komplexe Koeffizienten. FrequencyFlat
erwartet genau $N_\mathrm{tx}N_s$ Werte in tx/stream-Reihenfolge und
expandiert sie beim Erzeugen des Werttyps explizit auf alle N_sc
Subcarrier. Intern existiert damit kein implizites Broadcasting und jeder
Zugriff besitzt dieselben drei Dimensionen. Alle Real- und Imaginärteile
müssen endlich sein.

Für jeden Subcarrier gilt verpflichtend

$$
\lVert\mathbf W[k]\rVert_F^2
=\sum_{t=0}^{N_\mathrm{tx}-1}\sum_{s=0}^{N_s-1}|w_{t,s}[k]|^2
=N_s.
$$

Die absolute Prüftoleranz beträgt 10^{-10}. Eine verletzte Norm wird
abgelehnt und nicht stillschweigend korrigiert. Mit
$\mathrm E[\mathbf x\mathbf x^H]=\mathbf I$ lautet das erweiterte
Signalmodell

$$
\mathbf y[k]=
\sqrt{\frac{P}{N_s}}\,\mathbf H[k]\mathbf W[k]\mathbf x[k]
+\mathbf n[k].
$$

Damit bleibt P die gesamte Sendeleistung und

$$
\mathbf H_\mathrm{eff}[k]=\mathbf H[k]\mathbf W[k]
\in\mathbb C^{N_\mathrm{rx}\times N_s}.
$$

Der bestehende MMSE-Kern wird ohne neuen Detektortyp auf
$\mathbf H_\mathrm{eff}$ angewendet. Mit
$\alpha=\rho/N_s$ gilt weiterhin

$$
  \mathbf E[k]=
  \left(\mathbf I_{N_s}+\alpha
  \mathbf H_\mathrm{eff}^H[k]\mathbf H_\mathrm{eff}[k]\right)^{-1},
  \qquad
  \gamma_s[k]=\frac{1}{E_{s,s}[k]}-1.
$$

SuMimoLinkModel, HeSuMimoLinkAbstraction und der Spectrumadapter besitzen
dafür explizite Precoding-Pfade. Ihre bisherigen Überladungen ohne W[k]
bleiben bestehen und rechnen unverändert direkt mit H[k]. Insbesondere nimmt
der bytegleiche 1x1-IPC-Vertrag kein neues Feld auf: Sionna liefert weiterhin
ausschließlich H[k], W[k] existiert nur in ns-3.

SionnaMimoConfiguration enthält optional precodingMatrix. Validate prüft vor
jeder Helper-Zustandsänderung die TX-Dimension, die Gleichheit von maxNss und
W-Spalten sowie $N_s\leq N_\mathrm{rx}$. Apply ersetzt den Adapterzustand
entweder durch den vollständig validierten Precoder oder löscht ihn explizit,
sodass erneutes Anwenden einer Phase-2-Konfiguration kein altes W[k]
zurücklässt. Am Laufzeitpfad werden zusätzlich CFR-Subcarrierzahl und beide
CFR-Antennendimensionen geprüft, bevor ErrorRateModel- oder Empfangszustand
verändert wird. WifiTxVector::GetNss muss bei vorhandenem W[k] exakt dessen
Spaltenzahl, andernfalls weiterhin exakt N_tx sein.

Unterphase 3.1 implementiert benutzergegebenes festes Precoding, Unterphase
3.2 ergänzt die CFR-abhängige MISO-MRT-Fabrik und Unterphase 3.3 die
deterministische allgemeine SVD-Fabrik mit gleicher oder wassergefüllter
räumlicher Leistung. Alle erzeugen unveränderliche W[k]-Schnappschüsse, die
anschließend denselben H[k]W[k]-Pfad durchlaufen. CSI-Rückmeldung, CSI-Alterung,
automatische Laufzeit-Beamformingmodi und MU-MIMO bleiben außerhalb dieses
Vertrags.

5.10 EIN-STREAM-MRT/MRC DER UNTERPHASE 3.2

Für den eindeutig definierten MISO-Fall $N_\mathrm{rx}=1$ sei die CFR je
Subcarrier der Zeilenvektor

$$
  \mathbf h[k]\in\mathbb C^{1\times N_\mathrm{tx}}.
$$

Bei $\lVert\mathbf h[k]\rVert_2>0$ erzeugt
MimoPrecodingMatrix::MaximumRatioTransmission aus perfekter CFR genau einen
Precodervektor

$$
  \mathbf w_\mathrm{MRT}[k]
  =\frac{\mathbf h^H[k]}{\lVert\mathbf h[k]\rVert_2},
  \qquad
  \lVert\mathbf w_\mathrm{MRT}[k]\rVert_2^2=1=N_s.
$$

Damit erfüllt das Ergebnis direkt den Leistungsvertrag aus Abschnitt 5.9 und
der effektive Kanal ist

$$
  \mathbf h[k]\mathbf w_\mathrm{MRT}[k]
$$

$$
  =\lVert\mathbf h[k]\rVert_2.
$$

Der unveränderte Ein-Stream-MMSE-Kern liefert folglich

$$
  E[k]=\frac{1}{1+\rho\lVert\mathbf h[k]\rVert_2^2},
  \qquad
  \gamma[k]=\rho\lVert\mathbf h[k]\rVert_2^2.
$$

Die Normierung erfolgt unabhängig für jeden Subcarrier; das zurückgegebene W[k] besitzt deshalb dieselbe sc/tx/stream-Reihenfolge wie jeder feste Precoder. Eine gemeinsame Phasendrehung $\mathbf h'[k]=e^{j\phi}\mathbf h[k]$ führt zu $\mathbf w'[k]=e^{-j\phi}\mathbf w[k]$ und lässt den effektiven Kanal sowie die SINR invariant. Besitzt auch nur ein Subcarrier Norm null, wird der gesamte Erzeugungsaufruf eindeutig mit std::invalid_argument abgelehnt. Es wird weder eine beliebige Strahlrichtung gewählt noch Energie erzeugt. Nicht endliche CFR-Koeffizienten werden ebenfalls abgelehnt.

Für $N_\mathrm{tx}=1$ ist $\mathbf H[k]=\mathbf h[k]\in\mathbb C^{N_\mathrm{rx}\times1}$. Dann ist die Richtung des linearen MMSE-Kombinierers kollinear zu $\mathbf h[k]$ und die bestehende Lösung reduziert sich ohne neuen Empfängertyp auf MRC:

$$
  \gamma[k]=\rho\,\mathbf h^H[k]\mathbf h[k]
  =\rho\lVert\mathbf h[k]\rVert_2^2.
$$

Im 1x1-Fall entfernt MRT nur die Kanalphase; der Betrag und damit das bisherige skalare Ergebnis bleiben exakt erhalten. Für $N_\mathrm{tx}>1$ und $N_\mathrm{rx}>1$ weist die MRT-Funktion den Aufruf bewusst ab. Eine optimale Ein-Stream-Richtung in diesem allgemeinen Fall ist der stärkste rechte Singulärvektor und gehört ausschließlich in Unterphase 3.3.

Der ideale numerische Aufruf lautet:

```
    const auto mrt =
    MimoPrecodingMatrix::MaximumRatioTransmission(channelAtCsiTime);
    const auto result = SuMimoLinkModel::Calculate(channelAtCsiTime, mrt, rho);
```

Das Ergebnis ist ein unveränderlicher W[k]-Schnappschuss für die übergebene perfekte CFR. Unterphase 3.2 führt absichtlich keinen automatischen Helper-/SpectrumWifiPhy-Beamformingmodus, keine CSI-Zeitachse und keine periodische Aktualisierung ein. Der erzeugte Precoder kann wie jeder feste 2x1-Precoder durch die bereits validierten Pfade aus Unterphase 3.1 geführt werden; die darauf aufbauende end-to-end Modusauswahl ist nun in Abschnitt 6.7 als Teil der abgeschlossenen Unterphase 3.5 beschrieben.

5.11 DETERMINISTISCHES SVD-PRECODING UND LEISTUNGSALLOKATION DER UNTERPHASE 3.3

Für jeden Subcarrier berechnet MimoPrecodingMatrix::SingularValueDecomposition aus der explizit übergebenen perfekten CFR eine dünne Jacobi-SVD

$$
  \mathbf H[k]=\mathbf U[k]\boldsymbol\Sigma[k]\mathbf V^H[k],
$$

$$
  \qquad
  \sigma_0[k]\geq\sigma_1[k]\geq\ldots\geq0.
$$

Die Streamzahl wird nicht aus dem Kanal erraten, sondern als N_s explizit übergeben und muss

$$
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})
$$

erfüllen. Der numerische Rang je Subcarrier ist

$$
  r[k]=\#\left\{i:\sigma_i[k]>
  \tau_\mathrm{rank}\sigma_0[k]\right\}.
$$

Der dokumentierte Standard ist $\tau_\mathrm{rank}=10^{-12}$; ein benutzergegebener Wert muss endlich und strikt in (0,1) liegen. Gilt auf auch nur einem Subcarrier $r[k]<N_s$, wird der gesamte Fabrikaufruf mit Subcarrierindex, geforderter Streamzahl und gefundenem Rang abgelehnt. Es gibt weder eine versteckte Regularisierung noch eine stille Verringerung von N_s. Nicht endliche CFR-Koeffizienten werden vor der Zerlegung zurückgewiesen.

Die von Eigen absteigend gelieferten Singularwerte erhalten zusätzlich eine explizite deterministische Vektorkonvention. Zwei Werte derselben Zerlegung gelten bei

$$
  |\sigma_i-\sigma_j|\leq10^{-12}\sigma_0
$$

als numerisch gleich. Für jede solche entartete Gruppe wird nicht die vom SVD-Algorithmus beliebig gewählte Basis übernommen. Stattdessen bildet ns-3 den eindeutigen Unterraumprojektor $\mathbf P=\mathbf V_g\mathbf V_g^H$, projiziert die kanonischen TX-Koordinatenachsen und orthonormalisiert sie deterministisch. Je Schritt wird die Achse mit der größten Residualnorm gewählt; numerisch gleiche Kandidaten verwenden den kleinsten TX-Index. Abschließend wird jeder Singulärvektor so phasengedreht, dass sein betragsgrößter Eintrag reell und nicht negativ ist; bei gleichem Betrag gewinnt ebenfalls der kleinste TX-Index. Damit sind Reihenfolge, Vorzeichen beziehungsweise komplexe Phase und die Basis bei gleichen Singularwerten vollständig festgelegt.

  GLEICHE LEISTUNG ALS STANDARD

Im Standardmodus MimoPowerAllocationMode::EQUAL_POWER gilt für alle gewählten Moden

$$
  f_s[k]=\frac{1}{N_s},
  \qquad
  \mathbf W[k]=[\mathbf v_0[k],\ldots,\mathbf v_{N_s-1}[k]].
$$

Damit besitzt jede Spalte Norm eins und der Vertrag aus Abschnitt 5.9 bleibt unverändert:

$$

$$
  \lVert\mathbf W[k]\rVert_F^2=N_s.
$$

Die SVD-Moden sind orthogonal. Der unveränderte MMSE-Kern liefert deshalb ohne Inter-Stream-Interferenz

$$
  \gamma_s[k]=\frac{\rho}{N_s}\sigma_s^2[k].
$$

Der 1x1-Grenzfall verwendet den einzigen rechten Singulärvektor mit Einheitsnorm und reproduziert das bisherige skalare Ergebnis exakt. Die Optionen verwenden für die normierten Leistungsdiagnosen standardmäßig $\rho=1$ und $N=1$; die gleichmäßige Richtungsauswahl ist davon unabhängig.

  OPTIONALE RÄUMLICHE WASSERFÜLLUNG

Wasserfüllung wird nur durch die explizite Auswahl MimoPowerAllocationMode::WATER_FILLING aktiviert. Sie verteilt die Leistung ausschließlich über die N_s bereits gewählten räumlichen SVD-Moden eines Subcarriers. Zwischen Subcarriern wird keine Leistung verschoben; spektrale Maske und Gesamtleistung je Subcarrier bleiben daher unverändert. Mit Rauschleistung $N>0$, Referenz-SNR $\rho>0$, Gesamtleistung $P=\rho N$ und Modengewinn $g_s[k]=\sigma_s^2[k]$ lauten die Leistungsanteile

$$
  f_s[k]=\left(
    \nu[k]-\frac{1}{\rho g_s[k]}
  \right)^+,
  \qquad
  \sum_{s=0}^{N_s-1}f_s[k]=1.
$$

Der aktive Modensatz und der normierte Wasserstand $\nu[k]$ werden deterministisch aus den absteigend sortierten Moden bestimmt. Eine Mode am exakten Aktivierungsschwellwert bleibt inaktiv. Der Precoder skaliert danach

$$
  \mathbf w_s[k]=\sqrt{N_s f_s[k]}\,\mathbf v_s[k].
$$

Somit gilt auch bei inaktiven Nullspalten exakt

$$
  \sum_s\lVert\mathbf w_s[k]\rVert_2^2
  =N_s\sum_s f_s[k]=N_s.
$$

Die physikalisch ausgewiesene Modenleistung und der Wasserstand sind

$$
  p_s[k]=P f_s[k],
  \qquad
  \mu[k]=P\nu[k].
$$

Für jede aktive Mode erfüllen sie die KKT-Bedingung

$$

$$
  p_s[k]+\frac{N}{g_s[k]}=\mu[k],
\]

während inaktive Moden keinen negativen Leistungsanteil erhalten. Eine
abschließende Rundungskorrektur wird ausschließlich auf den letzten aktiven
Anteil angewendet, sodass die Anteile wieder exakt zur normierten
Gesamtleistung summieren. Alle Gewinne, Anteile und Wasserstände werden auf
Endlichkeit und Nichtnegativität geprüft.

MimoSvdPrecodingResult hält den festen Precoder sowie pro Subcarrier
numerischen Rang, gewählte Singularwerte, aktive Modenzahl, Leistungsanteile,
absolute Modenleistungen, deren Summe und den Wasserstand. Zusätzlich werden
Rauschleistung, Referenz-SNR, Gesamtleistung und Rangtoleranz unverändert
ausgewiesen. Ein idealer Aufruf mit Wasserfüllung lautet:

```
    MimoSvdPrecodingOptions options;
    options.powerAllocation = MimoPowerAllocationMode::WATER_FILLING;
    options.referenceSnrLinear = rho;
    options.noisePower = noisePower;

    const auto svd = MimoPrecodingMatrix::SingularValueDecomposition(
    channelAtCsiTime, numStreams, options);
    const auto result = SuMimoLinkModel::Calculate(
    channelAtCsiTime, svd.GetPrecodingMatrix(), options.referenceSnrLinear);
```

Wie MRT erzeugt auch SVD ausschließlich einen unveränderlichen perfekten-CSI-
W[k]-Schnappschuss. Unterphase 3.3 aktiviert weder SU_BEAMFORMING im Helper
noch automatische Aktualisierung, Feedback, CSI-Zeitbezug oder einen neuen
SpectrumWifiPhy-Pfad. Der vorhandene feste-W[k]-Vertrag ist die einzige
Integration; Unterphase 3.5 verwendet genau diesen Vertrag nun für die
benutzerseitige end-to-end Modusauswahl, ohne die SVD- oder
Leistungsallokationssemantik zu verändern.

5.12 QUANTISIERTE HE-/EHT-SU-RÜCKMELDUNG UND CSI-ALTERUNG DER UNTERPHASE 3.4

UNTERSTÜTZTER FEEDBACKUMFANG UND ABGRENZUNG

MimoCompressedBeamformingFeedback bildet die semantische komprimierte
Beamformingmatrix $\mathbf V[k]$ aus der HE-/EHT-SU-Rückmeldung ab. IEEE
802.11ax beschreibt diese Matrix als geordnete Folge von
ϕ-Phasenwinkeln und reellen ψ-Givens-Winkeln. EHT übernimmt
dieselbe komprimierte Darstellung mit erweiterten Grenzen für N_c und
N_r; dies ist auch in der öffentlichen
[IEEE-802.11be-Beamforming-Korrespondenz](https://www.ieee802.org/11/email/stds-802-11-tgbe/msg01987.html)
festgehalten. Die öffentlich gelisteten
[HE-Comment-Resolution-Dokumente](https://www.ieee802.org/11/Reports/tgax_update.htm)
verweisen auf die HE Compressed Beamforming Report- und
SU-MIMO-Beamforming-Klauseln. Die Implementierung unterstützt daraus bewusst
folgenden schmalen Vertrag:

- ausschließlich Single-User-Feedback für eine semi-unitäre
  Gleichleistungs-SVD-Matrix mit $1\leq N_s\leq N_\mathrm{tx}$;
- HE mit höchstens acht und EHT mit höchstens 16 TX-Antennen;
- das grobe SU-Codebook mit $b_\phi=4,\ b_\psi=2$ sowie das feine
  SU-Codebook mit $b_\phi=6,\ b_\psi=4$;
- eine Matrix je modelliertem Subcarrier und einen unquantisierten
  Forschungsgrenzfall.

Der Typ ist absichtlich kein serialisierter IEEE-802.11-Action-Frame.
MIMO-Control-, Average-SNR-/CQI-Felder, MAC-Header, Segmentierung,
Subcarriergruppierung N_g, Interpolation, RU-/Partial-Bandwidth-Auswahl,

MU-Exclusive-Feedback und die feineren MU-Codebooks sind nicht implementiert. Wassergefülltes W[k] wird ebenfalls nicht als $\mathbf V[k]$ ausgegeben, weil dessen Spalten nicht semi-unitär sind; die Richtungsrückmeldung und eine separate Leistungsallokation werden nicht fälschlich vermischt. Damit wird weder MU-MIMO noch die Laufzeitintegration aus Unterphase 3.5 vorweggenommen.

GIVENS-DARSTELLUNG UND VERBINDLICHE WINKELREIHENFOLGE

Sei $m=\min(N_s,N_\mathrm{tx}-1)$. Für jede Stufe $i=0,\ldots,m-1$ enthält $\mathbf D_i$ auf den Diagonaleinträgen $i,\ldots,N_\mathrm{tx}-2$ die Phasen $\exp(j\phi_{r,i})$; der letzte Eintrag und die Einträge vor i sind eins. $\mathbf G_{l,i}^{T}(\psi_{l,i})$ ist die reelle Rotation, deren (i,l)-Block

$$
\begin{bmatrix}
\cos\psi_{l,i} & -\sin\psi_{l,i}\\
\sin\psi_{l,i} & \cos\psi_{l,i}
\end{bmatrix}
$$

lautet. Die Rekonstruktion verwendet exakt

$$
\widehat{\mathbf V}[k]
=
\left[
\prod_{i=0}^{m-1}
\mathbf D_i
\left(
\mathbf G_{N_\mathrm{tx}-1,i}^{T}
\cdots
\mathbf G_{i+1,i}^{T}
\right)
\right]
\mathbf I_{N_\mathrm{tx}\times N_s}.
$$

Die flache Reihenfolge je Subcarrier ist für jede Stufe zuerst

$$
\phi_{i,i},\ldots,\phi_{N_\mathrm{tx}-2,i}
$$

und danach

$$
\psi_{i+1,i},\ldots,\psi_{N_\mathrm{tx}-1,i}.
$$

Die getrennten ganzzahligen ϕ- und ψ-Arrays sind innerhalb ihres Typs subcarrier-major. Die Zahl der Winkel jedes Typs lautet

$$
A =
\sum_{i=0}^{m-1}(N_\mathrm{tx}-i-1).
$$

Encode entfernt die für die Linkmetrik irrelevante Phase jeder SVD-Spalte deterministisch, extrahiert danach die idealen Winkel stufenweise und

quantisiert nur diese Winkel. FromQuantizedAngles überspringt die Extraktion und erzeugt aus bekannten Integerindizes eine bitgenaue Referenzdarstellung. Reconstruct ist davon getrennt und verwendet ausschließlich die gespeicherten Indizes. Die idealen SVD-, Feedback- und Rekonstruktionsobjekte bleiben damit einzeln prüfbar.

DETERMINISTISCHE QUANTISIERUNG UND GRENZWERTE

Für $\phi\in[0,2\pi)$ und 2^{b_ϕ} Zellen gelten

$$
q_\phi =
\left\lfloor
  \frac{2^{b_\phi}\phi}{2\pi}
\right\rfloor,
\qquad
\widehat\phi =
\frac{2\pi(q_\phi+\tfrac12)}{2^{b_\phi}}.
$$

Vor der Quantisierung wird ϕ modulo 2π normalisiert; der obere Rand 2π fällt damit eindeutig auf Index null. Für $\psi\in[0,\pi/2]$ gilt

$$
q_\psi =
\min\left(
  2^{b_\psi}-1,
  \left\lfloor
  \frac{2^{b_\psi+1}\psi}{\pi}
  \right\rfloor
\right),
\qquad
\widehat\psi =
\frac{\pi(q_\psi+\tfrac12)}{2^{b_\psi+1}}.
$$

Damit rekonstruieren auch die minimalen und maximalen Indizes stets die Zellmitte und nie einen mehrdeutigen periodischen Rand. Nicht passende Indexanzahlen und Werte außerhalb der gewählten Bitbreite werden vor der Rekonstruktion mit std::invalid_argument abgelehnt. Der unquantisierte Modus speichert das bereits validierte feste W[k] direkt, rekonstruiert es koeffizientenexakt und weist null Feedbackbits aus; null bedeutet hier „kein endlicher Drahtumfang modelliert“ und nicht ein kostenloses Standardfeedback.

Der reine Winkelumfang ohne PHY-/MAC-Header und ohne CQI ist

$$
B_\mathrm{fb}
=
N_\mathrm{sc} A (b_\phi+b_\psi).
$$

Für eine 2x1-Matrix sind dies je Subcarrier sechs Bit im groben und zehn Bit im feinen SU-Codebook. Eine 3x2-Matrix besitzt je Typ drei Winkel und benötigt im feinen Profil 30 Bit je Subcarrier.

LEISTUNGSRENORMIERUNG UND DIAGNOSEGRÖSSEN

Obwohl das Produkt idealer Rotationen semi-unitär ist, wird jede quantisiert

rekonstruierte Matrix vor dem bestehenden festen-W[k]-Konstruktor explizit erneut normalisiert. Mit

$$
  P_\mathrm{vor}[k]
  =
  \lVert\widehat{\mathbf V}[k]\rVert_F^2,
  \qquad
  a[k]=\sqrt{\frac{N_s}{P_\mathrm{vor}[k]}},
  \qquad
  \mathbf W_q[k]=a[k]\widehat{\mathbf V}[k]
$$

gilt anschließend wieder

$$
  \lVert\mathbf W_q[k]\rVert_F^2=N_s.
$$

MimoFeedbackReconstruction weist $P_\mathrm{vor}[k]$, $a[k]$ und die endgültige Leistung getrennt aus. Nullleistung, nicht endliche Werte oder ein Ergebnis außerhalb des festen-W[k]-Vertrags werden abgelehnt.

MimoFeedbackDiagnostics meldet neben B_fb zwei phaseninvariante Fehler je Subcarrier. Für normierte ideale und rekonstruierte Spalten sind dies der mittlere Spalten-Chordalfehler

$$
  e_\mathrm{ch}[k]
  =
  \sqrt{
    \frac1{N_s}\sum_s
    \left(
    1-
    \frac{
    |\mathbf w_s^H[k]\mathbf w_{q,s}[k]|^2
    }{
    \lVert\mathbf w_s[k]\rVert_2^2
    \lVert\mathbf w_{q,s}[k]\rVert_2^2
    }
    \right)
  }
$$

und der RMS-Koeffizientenfehler nach optimaler gemeinsamer Phasenausrichtung jeder Spalte. Zusätzlich stehen die Mittelwerte über alle Subcarrier bereit. Damit kann der Fehler ohne willkürliche SVD-Spaltenphase verglichen werden. Auf den Referenzvektoren ist das feine Codebook nicht schlechter als das grobe.

  EXPLIZITE CSI-ZEITACHSE

MimoCsiAgingModel::Select erhält ausschließlich die zur Auswahl verwendete Schätzung

$$
  \widehat{\mathbf H}[k,t_\mathrm{csi}]
$$

sowie t_csi, t_tx und eine MimoCsiSelectionConfiguration. Mit Aktualisierungsintervall $T_u\geq0$ und

Feedbackverzögerung $d_\mathrm{fb}\geq0$ ist der neueste zum Sendezeitpunkt verfügbare Schätzzeitpunkt verbindlich

$$
  t_\mathrm{csi}
  =
  \begin{cases}
    t_\mathrm{tx}-d_\mathrm{fb}, & T_u=0,\\
    T_u\left\lfloor
    \dfrac{t_\mathrm{tx}-d_\mathrm{fb}}{T_u}
    \right\rfloor, & T_u>0.
  \end{cases}
$$

Ein Sendezeitpunkt vor d_fb besitzt noch kein verfügbares Feedback. Ein explizit übergebener Zeitstempel, der nicht der Gleichung entspricht, wird abgelehnt. Das ausgewiesene CSI-Alter ist

$$
  \Delta t_\mathrm{CSI}=t_\mathrm{tx}-t_\mathrm{csi}.
$$

$T_u=0$ und $d_\mathrm{fb}=0$ bilden den perfekten Nullalter-Grenzfall. Select berechnet aus $\widehat{\mathbf H}[k,t_\mathrm{csi}]$ zuerst das deterministische Gleichleistungs-SVD-W[k], danach Feedback und Rekonstruktion. CalculateLink verlangt dagegen die davon getrennt übergebene aktuelle CFR

$$
  \mathbf H[k,t_\mathrm{tx}]
$$

und berechnet ausschließlich $\mathbf H[k,t_\mathrm{tx}]\mathbf W_q[k]$. Der CSI-Schnappschuss wird in diesem Schritt nicht gespeichert oder wiederverwendet. Derselbe feste Precoder kann an HeSuMimoLinkAbstraction übergeben werden; dadurch verwenden auch EESM und PER explizit die aktuelle CFR. Eine kontrollierte orthogonale Subraumrotation liefert im Test mit altem W null SINR und eine entsprechend schlechtere PER, während ein statischer Kanal trotz positiven Alters unverändert bleibt.

MimoCsiStateIdentity ist der vollständige wertbasierte Schlüssel des Auswahlzustands. Er enthält gerichtete TX- und RX-Node-IDs, beide Arraygeometrien einschließlich Abstand, Muster, Polarisation und Orientierung, den Precodingmodus, HE-/EHT-Feedbackprofil und t_csi. Vertauschte Richtung, eine geänderte Arraykonfiguration, ein anderes Profil oder ein anderer CSI-Zeitstempel sind damit verschiedene Zustände. Diese Identität ist von der bestehenden rohen Sionna-PropagationCache-Ablage getrennt und verhindert, dass ein späterer Laufzeitcache versehentlich Precoderzustände verschiedener Richtungen oder Konfigurationen teilt.

Der ideale Aufruf ohne Quantisierung lautet:

```
    MimoCsiSelectionConfiguration configuration;
    configuration.numStreams = numStreams;
    configuration.feedback.codebook =
    MimoFeedbackCodebook::UNQUANTIZED;
    configuration.timing.updateInterval = MilliSeconds(5);
    configuration.timing.feedbackDelay = MilliSeconds(2);

    const auto selection = MimoCsiAgingModel::Select(
    channelEstimateAtCsiTime,
```

```
    csiTimestamp,
    transmissionTimestamp,
    configuration);
    const auto currentLink = selection.CalculateLink(
    channelAtTransmissionTime,
    referenceSnrLinear);
```

Der Vertrag bleibt bewusst keine automatische Simulationstaktung. Unterphase 3.5 bindet ihn nun durch einen expliziten SelectPrecoding-Aufruf an SionnaMimoHelper und SpectrumWifiPhy: Der Anwender übergibt CSI- und Sendezeitpunkt, der Helper hält das unveränderliche Ergebnis und Apply nutzt anschließend den vorhandenen festen-W[k]-Pfad. Periodische Ereignisse oder MAC-Feedbackframes werden dadurch nicht vorgetäuscht.

5.13 FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL DER UNTERPHASE 4.2

`DlMuMimoPrecodingMatrix` bettet den unveränderten, leistungsvalidierten `MimoPrecodingMatrix`-Tensor ein. Der Index eines Eintrags im Zuordnungsvektor ist die globale Precoder-Spalte c; der Eintrag

$$
a_c=(u,s_u)
$$

nennt die stabile Nutzer-ID u und den nutzerlokalen Stream s_u. Die lokalen Streamindizes jedes Nutzers beginnen bei null und sind lückenlos. Jeder Nutzer des Phase-4.1-Snapshots erhält mindestens einen Stream, und kein Stream darf auf einen nicht vorhandenen Nutzer zeigen.

Die Phase-3-Normierung gilt jetzt über alle gleichzeitig gesendeten Spalten:

$$
\Vert W[k]\Vert_F^2=N_{s,\mathrm{ges}}.
$$

Mit der expliziten Spaltenleistung

$$
q_c[k]=\sum_t |W_{t,c}[k]|^2,\qquad
\sum_c q_c[k]=N_{s,\mathrm{ges}}
$$

beträgt der Anteil der gesamten AP-Leistung für Spalte c genau $q_c[k]/N_{s,\mathrm{ges}}$. Gleiche Spaltennormen ergeben gleiche Leistung; abweichende Nutzer- oder Streamgewichte stehen sichtbar in $W[k]$, ohne die Gesamtleistung zu ändern.

Für Nutzer u wird zunächst der vollständige effektive Kanal gebildet:

$$
G_u[k]=H_u[k]W[k]
  =\left[g_{u,0}[k],\ldots,g_{u,N_{s,\mathrm{ges}}-1}[k]\right].
$$

Die nutzerspezifische Referenz-SNR ρ_u enthält den getrennten Wideband-Pfadverlust und die Empfängerrauschleistung. Für jede globale Spalte gilt derselbe Symbolleistungsfaktor

$$
\alpha_u=\frac{\rho_u}{N_{s,\mathrm{ges}}}.
$$

\]

Der lineare Empfänger verwendet die vollständige Kovarianz aller gewünschten
und fremden Spalten,

\[
  R_u[k]=I+\alpha_u G_u[k]G_u^H[k],
  \qquad
  f_{u,c}[k]=R_u^{-1}[k]g_{u,c}[k].
\]

Die Skalierung von $f_{u,c}$ ist für die SINR ohne Bedeutung. Der ausgegebene
gewünschte Anteil und die Inter-User-Interferenz nach Kombination lauten

\[
  D_{u,c}[k]=\alpha_u|f_{u,c}^H[k]g_{u,c}[k]|^2,
\]

\[
  I_{u,c}^{\mathrm{MU}}[k]
    =\alpha_u\sum_{d:a_d=(v,\cdot),\,v\ne u}
    |f_{u,c}^H[k]g_{u,d}[k]|^2.
\]

Mit dem analogen Intra-User-Anteil über die anderen Streams desselben Nutzers
und $N_{u,c}[k] = \|f_{u,c}[k]\|_2^2$ folgt

\[
  \gamma_{u,c}[k]
    =\frac{D_{u,c}[k]}
    {N_{u,c}[k]+I_{u,c}^{\mathrm{intra}}[k]
    +I_{u,c}^{\mathrm{MU}}[k]}.
\]

Für genau einen Nutzer ist dies algebraisch derselbe lineare MMSE-Vertrag wie
in Phase 3. Das gilt auch für den 1x1-Grenzfall und für null SNR
beziehungsweise einen verschwindenden effektiven Kanal.

`HeDlMuMimoLinkAbstraction` komprimiert anschließend ausschließlich die
SINR-Werte des jeweiligen Nutzers:

\[
  \gamma_{\mathrm{eff},u}
   =-\beta_{m_u}\ln\left(
    \frac{1}{N_{\mathrm{sc}}N_{s,u}}
    \sum_{k,s_u}\exp\left(-\frac{\gamma_{u,s_u}[k]}{\beta_{m_u}}\right)
    \right).
\]

MCS m_u, EESM-β_{m_u}, `WifiTxVector` und Paketgröße gehören
explizit zum Nutzer. Die anschließende ErrorRateModel-Abfrage ist daher
getrennt und kann für zwei Nutzer unterschiedliche PER liefern.

Vor der Berechnung werden Snapshotrichtung und -ordnung, CSI-Zeitidentität,
Frequenzraster, CFR-/Precoderdimensionen, endliche Koeffizienten, die
vollständige Spaltenzuordnung, nutzerweise Stream-/RX-Grenzen sowie genau eine
SNR- und HE-Eingabe je Nutzer geprüft. Konstruktion und Berechnung sind
wertbasiert; eine Ablehnung verändert weder Snapshot noch Precoder oder
EESM-Konfiguration. Der Kern konsumiert den Phase-4.1-Snapshot nur lesend und
ändert weder Protocol Buffers, Batching noch Cache.

6. HIGH-LEVEL-API-VERTRAG BIS UNTERPHASE 3.5

6.1 TYPEN, BESITZ UND DEFAULTS

SionnaMimoHelper ist ein wertbasierter, simulationsunabhängiger Konfigurationshalter. Sein Konstruktor erzeugt einen gültigen SISO-Vertrag; der Konstruktor mit SionnaMimoConfiguration und SetConfiguration prüfen eine vollständige Kopie, bevor sie den gespeicherten Zustand ersetzen. Eine fehlgeschlagene Änderung lässt den bisherigen Zustand damit unverändert. Alle Validierungsfehler sind std::invalid_argument mit einem SionnaMimoHelper-Präfix und benötigen weder Simulator, Node noch eine ZMQ-Verbindung.

SionnaMimoArrayConfiguration enthält Zeilen, Spalten, vertikalen und horizontalen Abstand in Wellenlängen, Elementmuster und Polarisation. Die repräsentierten Sionna-Muster sind isotrop, dipole, half-wave dipole und TR 38.901. Vertikale und horizontale Polarisation erzeugen je Element einen, duale Polarisation zwei Antennenports. Für Zeilenzahl R, Spaltenzahl C und Polarisationsfaktor $p\in\{1,2\}$ gilt deshalb

$$
  N_\mathrm{array} = R C p.
$$

SionnaMimoOrientation speichert Azimut, Elevation und Rollwinkel in Grad. Die SionnaMimoConfiguration führt beide Arrays, den Modus, TX-/RX-Antennenzahl, maximale NSS, Empfänger, Normalisierung, Empfängerrauschzahl, Effective-SINR-Modell, explizite EESM-Betas und eine Map von ns-3-Node-ID auf Orientierung sowie optional MimoPrecodingMatrix oder eine explizite SionnaMimoBeamformingConfiguration zusammen. Die festgeschriebenen Defaults sind:

- Modus SISO, eine TX-Antenne, eine RX-Antenne und maximale NSS eins;
- TX-Array 1 x 1, Abstände 0,5 Wellenlängen, TR-38.901-Muster, vertikale Polarisation;
- RX-Array 1 x 1, Abstände 0,5 Wellenlängen, Dipolmuster, vertikale Polarisation;
- keine explizite knotenspezifische Orientierung; der API-Lesezugriff liefert dafür neutrale Nullwinkel, während Apply kein IPC-Feld erzeugt und der Server deshalb für alte und neue 1x1-Aufrufe exakt seine bisherige interne Orientierung [0, -180, 0] beibehält;
- linearer MMSE-Empfänger und gemeinsame Unit-Average-Power-Normalisierung über alle Subcarrier und Antennenpaare gemäß Abschnitt 4.3;
- 7 dB Empfängerrauschzahl entsprechend dem ns-3.40-WifiPhy-Default;
- EESM als HE-Effective-SINR-Modell, aber weiterhin keine impliziten MCS-Betas;
- weder ein fester Precoder noch Beamformingoptionen oder eine ausgewählte Precodingmomentaufnahme. Dadurch nehmen alle bisherigen 1x1- und offenen Mehrstromkonfigurationen weiterhin exakt den Phase-2-Pfad.

SetNodeOrientation, ClearNodeOrientation und GetNodeOrientation verwalten die knotenbezogenen Winkel. SetEesmBeta, HasEesmBeta und GetEesmBeta verwalten explizite lineare Betas für HE MCS 0 bis 11. SetPrecodingMatrix, ClearPrecodingMatrix, HasPrecodingMatrix und GetPrecodingMatrix verwalten den optionalen festen Precoder atomar. Die gespeicherte Konfiguration bleibt ein wertbasierter Vertrag und verändert Simulationsobjekte erst mit Apply.

6.2 UNTERSTÜTZTE BETRIEBSARTEN

GetSupportedModes und IsModeSupported melden genau drei akzeptierte Modi:

1. SISO mit genau einer TX- und einer RX-Antenne sowie NSS eins;

2. SU_MIMO_SPATIAL_MULTIPLEXING mit linearem MMSE-Empfänger. Ohne W[k]
   bleibt genau ein Stream je TX-Antenne aktiv; mit explizitem festem W[k]
   bestimmt dessen Spaltenzahl \(N_s\);
3. SU_BEAMFORMING mit explizit gewähltem MRT oder SVD. SelectPrecoding erzeugt
   aus der übergebenen CSI-Matrix genau eine unveränderliche Auswahl, die
   Apply anschließend über den vorhandenen festen-W[k]-Pfad installiert.

Festes benutzergegebenes Precoding bleibt bewusst
SU_MIMO_SPATIAL_MULTIPLEXING zugeordnet, weil es keine aus H[k] abgeleitete
Strahlformung implementiert. MRT ist weiterhin auf perfektes aktuelles
2x1-MISO-CSI und einen Stream begrenzt. SVD unterstützt Gleichleistung,
quantisiertes HE-/EHT-SU-Feedback und explizites CSI-Alter. Räumliche
Wasserfüllung ist mit unquantisiertem aktuellem CSI verfügbar; ihre
Kombination mit Feedbackquantisierung oder Alterung bleibt aufgrund des
semi-unitären Feedbackvertrags unzulässig. MU_MIMO bleibt der einzige
reservierte Modus und wird eindeutig abgelehnt. Ebenso werden der
repräsentierte Zero-Forcing-Empfänger, eine unnormalisierte CFR und Mutual
Information als PER-wirksames Effective-SINR-Modell abgelehnt. Die
MI-Kompression aus Abschnitt 5.4 bleibt eine Diagnosemetrik.

6.3 ZENTRALE KONSISTENZPRÜFUNG

Validate erzwingt gemeinsam die Grenzen von Sionna, ns-3-Wi-Fi und dem
Phase-1-Abstraktionsmodell:

- Arraydimensionen und Abstände sind endlich und positiv; die abgeleitete
  Portzahl stimmt exakt mit numTxAntennas beziehungsweise numRxAntennas
  überein;
- TX-/RX-Antennenzahlen und maxNss liegen im ns-3.40-Bereich 1 bis 8;
- SISO besitzt exakt die Dimensionen 1 x 1 und maxNss eins;
- SU-MIMO ohne W[k] erfüllt weiterhin
  \(2\leq N_\mathrm{tx}\leq N_\mathrm{rx}\leq8\) und
  \(N_\mathrm{ss,max}=N_\mathrm{tx}\);
- SU-MIMO mit W[k] erfüllt
  \(1\leq N_s=N_\mathrm{ss,max}\leq
  \min(N_\mathrm{tx},N_\mathrm{rx})\). TX- und Streamdimension stimmen
  exakt mit der Matrix überein; Subcarrierzahl, Koeffizienten und
  Leistungsnorm sind bereits Bestandteil des unveränderlichen
  MimoPrecodingMatrix-Vertrags;
- SU_BEAMFORMING besitzt mindestens zwei TX-Antennen, kein gleichzeitig
  benutzergegebenes W[k] und eine explizite MRT- oder SVD-Konfiguration;
- MRT verlangt genau eine RX-Antenne, einen Stream, unquantisiertes aktuelles
  CSI und unveränderte SVD-Standardoptionen;
- SVD prüft endliche positive Referenz-SNR und Rauschleistung, einen relativen
  Rangschwellwert in (0,1), HE/EHT und unquantisiertes, grobes oder feines
  SU-Codebook sowie nichtnegative CSI-Zeiten. Wasserfüllung schließt
  Quantisierung und positives CSI-Alter aus;
- jede Änderung der Konfiguration oder einer Knotenorientierung verwirft eine
  zuvor ausgewählte Momentaufnahme, damit Apply kein veraltetes W[k] nutzt;
- Azimut und Rollwinkel liegen endlich in [-180, 180] Grad, Elevation endlich
  in [-90, 90] Grad;
- die Rauschzahl ist endlich und nicht negativ;
- jedes vorhandene EESM-Beta gehört zu HE MCS 0 bis 11 und ist endlich und
  positiv.

ValidateForHeMcs ergänzt die nutzungsbezogene Vollständigkeitsprüfung: Für
Open-Loop-SU-MIMO und SU_BEAMFORMING muss das konkret verwendete MCS ein
explizites Beta besitzen. SISO verlangt wegen des unveränderten skalaren
Phase-1-Pfads weiterhin kein Beta. Damit werden keine unbelegten
Kalibrierwerte eingeführt.

6.4 GEMEINSAMER INTEGRATIONSAUFRUF

Apply erhält einen vorhandenen SionnaHelper, getrennte SpectrumWifiPhyHelper-Instanzen für Sender und Empfänger sowie den SionnaSpectrumPropagationLossModel-Adapter. Die Trennung der beiden PHY-Helper hält auch $N_\mathrm{tx}\ne N_\mathrm{rx}$ darstellbar. Apply validiert zuerst erneut den vollständigen gespeicherten Vertrag und verändert danach ausschließlich die explizit übergebenen Objekte:

```
    SionnaMimoHelper mimo(configuration);
    mimo.Apply(sionna, txPhy, rxPhy, spectrumModel);
```

Die Zuordnung ist eindeutig:

- txArray und rxArray werden auf die bestehenden ConfigureTransmitArray-/ConfigureReceiveArray-Aufrufe abgebildet;
- numTxAntennas beziehungsweise numRxAntennas setzen das Antennenzahlattribut des jeweiligen SpectrumWifiPhyHelper;
- maxNss setzt auf beiden Endpunkten MaxSupportedTxSpatialStreams und MaxSupportedRxSpatialStreams;
- receiverNoiseFigureDb setzt sowohl WifiPhy::RxNoiseFigure als auch den expliziten Rauschzahlwert des MIMO-Spectrumadapters;
- die vollständige EESM-Map ersetzt atomar die bisherige Adapterkonfiguration, sodass beim erneuten Anwenden keine veralteten MCS-Betas zurückbleiben;
- eine durch SelectPrecoding gespeicherte MRT-/SVD-Auswahl ersetzt den festen Adapter-Precoder vollständig und hat ausschließlich im Modus SU_BEAMFORMING Vorrang;
- precodingMatrix ersetzt den festen Adapter-Precoder vollständig. Fehlt der optionale Wert in einem Nicht-Beamformingmodus, wird ein zuvor gesetztes W[k] gelöscht und der Phase-2-Pfad wiederhergestellt.

Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung und EESM bleiben unverändert. Festes oder ausgewähltes W[k] wählt ausschließlich die bereits vorhandene $H[k]W[k]$-Überladung. Eine abweichende Empfänger-, Normalisierungs- oder Effective-SINR-Auswahl wird weiterhin vor Apply abgelehnt.

6.5 KNOTENORIENTIERUNG IM IPC- UND SERVERPFAD

NodeInfo besitzt das neue optionale Feld orientation mit Azimut, Elevation und Rollwinkel in Grad. SionnaHelper trägt nur explizit konfigurierte Knotenorientierungen ein. Der Python-Server prüft endliche Werte und dieselben Winkelgrenzen wie die High-Level-API, konvertiert Grad mit

$$
\boldsymbol\theta_\mathrm{rad}
= \frac{\pi}{180}\boldsymbol\theta_\mathrm{deg}
$$

und verwendet die drei Eulerwinkel beim Anlegen jedes Transmitter- und Receiver-Objekts im klassischen sowie im Look-ahead-Pfad. Das ausgelagerte PopulateMimoConfiguration schreibt Array- und Orientierungsfelder sowohl für Start als auch für den bytegenauen Integrationstest.

Fehlt orientation, verwendet der Server weiterhin exakt die bisherige Konstante [0, -180, 0]. Der Wert wird bewusst nicht als neue API-Orientierung interpretiert, sondern nur zur vollständigen Verhaltenskompatibilität alter Nachrichten beibehalten. Ein alter SionnaHelper-Aufruf konfiguriert weder Array- noch Orientierungsfelder und erzeugt deshalb weiterhin die bisherigen 1x1-Arrays und die bisherige Serverausrichtung.

6.6 BENUTZBARKEIT, BEISPIELE UND MIGRATION

Für eine 1x1-Verbindung reicht der Defaultvertrag. Er behält SISO, je eine TX-/RX-Antenne, NSS eins, 7 dB Rauschzahl, die bisherigen Arraymuster und den skalaren Legacy-Empfangspfad bei. Deshalb ist für diesen Pfad kein EESM-Beta erforderlich:

```
    SionnaMimoConfiguration configuration;
    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Das kleinste Vollrang-2x2-Beispiel verwendet zwei vertikale Elemente je Array, zwei offene Spatial Streams und ein vom Anwender kalibriertes Beta für den gewählten HE-MCS:

```
    SionnaMimoConfiguration configuration;
    configuration.mode = SionnaMimoMode::SU_MIMO_SPATIAL_MULTIPLEXING;
    configuration.txArray.numColumns = 2;
    configuration.rxArray.numColumns = 2;
    configuration.numTxAntennas = 2;
    configuration.numRxAntennas = 2;
    configuration.maxNss = 2;
    configuration.eesmBetaLinear.emplace(mcs, betaFromCalibration);

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Eine feste 2x1-Streamauswahl auf einem 2x2-Kanal ergänzt ausschließlich maxNss und den normalisierten Precoder:

```
    configuration.maxNss = 1;
    configuration.precodingMatrix = MimoPrecodingMatrix::FrequencyFlat(
    numSubcarriers, 2, 1, {{1.0, 0.0}, {0.0, 0.0}});

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Für frequenzselektives Precoding wird stattdessen der MimoPrecodingMatrix-Konstruktor mit $N_\mathrm{sc}N_\mathrm{tx}N_s$ Koeffizienten in sc/tx/stream-Reihenfolge verwendet. Beide Varianten durchlaufen dieselbe Validierung und denselben Adapterpfad.

ApplyForHeMcs führt zuerst ValidateForHeMcs aus. Ein ungültiger MCS oder ein fehlendes SU-MIMO-Beta bricht daher mit std::invalid_argument und einem SionnaMimoHelper-Präfix ab, bevor SionnaHelper, die beiden PHY-Helper oder der Spectrumadapter verändert werden. Apply bleibt für vorhandenen Code erhalten, wenn der verwendete MCS an dieser Stelle noch nicht bekannt ist.

Das Programm ns3_wifi_sumimo_minimal.cc enthält die vollständigen helperkonfigurierten 1x1-, offenen Vollrang-2x2-, Identitäts-Precoding- und 2x1-Streamauswahlszenarien. Es verbindet sie über eine feste normalisierte CFR mit dem echten UDP-/SpectrumWifiPhy-Pfad und behält zusätzlich den Phase-1-Rang-1-Vergleich:

```
    ./ns3 run "ns3_wifi_sumimo_minimal \
    --mcs=5 --referenceSnrDb=20 --packetSize=1200 \
    --eesmBetaLinear=<kalibrierter-linearer-Wert>"
```

Der feste Kanal ersetzt in diesem reproduzierbaren Beispiel nur die Kanalquelle. Array, Antennenzahlen, NSS, Rauschzahl und EESM werden nicht am

Adapter oder PHY gesetzt, sondern ausschließlich mit SionnaMimoHelper.

Für die Migration eines bestehenden Programms werden die manuellen Aufrufe ConfigureTransmitArray, ConfigureReceiveArray, die PHY-Attribute Antennas, MaxSupportedTxSpatialStreams, MaxSupportedRxSpatialStreams und RxNoiseFigure sowie SetReceiverNoiseFigureDb/SetEesmBeta am Spectrumadapter durch genau eine SionnaMimoConfiguration und ApplyForHeMcs ersetzt. SionnaHelper::Configure, SetMode, Start und Destroy, der PropagationCache sowie die normale SpectrumChannel-Verdrahtung bleiben unverändert. Ein Programm, das weiterhin nur SionnaHelper verwendet, nimmt damit unverändert den bisherigen 1x1-Defaultpfad.

Funktional unterstützt der Stand bis Unterphase 3.5 SISO, offenen SU-MIMO, festes benutzergegebenes Precoding und explizites SU-Beamforming mit MRT oder SVD. Für festes W[k] gilt weiterhin $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$. Die MRT-Fabrik erzeugt für eine explizit übergebene perfekte MISO-CFR ein normiertes Ein-Stream-W[k]. Die SVD-Fabrik erzeugt aus einer allgemeinen perfekten CFR ein festes W[k] für explizit gewählte, numerisch tragfähige Eigenmoden; gleiche Leistung bleibt Standard und räumliche Wasserfüllung ist eine explizite Option. Beide Fabriken verwenden anschließend unverändert den festen Precodingpfad. Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung der CFR und EESM bleiben unverändert. MimoCompressedBeamformingFeedback, MimoFeedbackReconstruction und MimoFeedbackDiagnostics bilden nun den getrennten Feedbackpfad. MimoCsiAgingModel liefert einen festen Precoder-Schnappschuss mit explizitem Zeitbezug und zwingt für die Linkberechnung eine separate Übertragungskanal-Matrix. Der Helper kapselt diese wertbasierten Ergebnisse unverändert; sie bleiben auch ohne Simulator testbar. Typische eindeutige Fehler sind:

- SionnaMimoHelper: open-loop SU-MIMO uses one stream per TX antenna bei widersprüchlichem maxNss;
- SionnaMimoHelper: SU beamforming requires an explicit MRT or SVD configuration bei fehlender Auswahlmethode;
- SionnaMimoHelper: water-filling cannot be combined with quantized feedback or CSI aging für nicht darstellbare Vertragskombinationen;
- SionnaMimoHelper: SU beamforming must select a channel-dependent precoder before Apply bei fehlendem CSI-Auswahlschritt;
- SionnaMimoHelper: ... requires an explicit EESM beta for HE MCS ... bei fehlender Kalibrierung für Open Loop oder Beamforming;
- SionnaMimoHelper: maximum NSS must equal the fixed-precoding stream dimension bei widersprüchlicher Helper-Konfiguration;
- WifiTxVector NSS must equal the transmitted stream dimension, falls WifiTxVector und W-Spalten beziehungsweise der alte offene TX-Pfad widersprüchlich sind;
- MimoPrecodingMatrix: MRT requires exactly one RX antenna beziehungsweise ... a non-zero channel on every subcarrier für mehrdeutige oder nicht normierbare MRT-Eingaben;
- MimoPrecodingMatrix: SVD stream count must be in ... beziehungsweise ... requested ... streams but subcarrier ... has numerical rank ... für ungültige Streamwahl oder Rangverlust sowie eindeutige Meldungen für nicht endliche CFR-, SNR-, Rausch- und Allokationseingaben.
- MimoCsiFeedback: SU compressed feedback requires a semi-unitary equal-power precoder beziehungsweise ... angle index exceeds the selected codebook width für nicht darstellbare Leistung oder ungültige Indizes;
- MimoCsiFeedback: CSI timestamp does not match update interval and feedback delay beziehungsweise ... no CSI feedback is available ... für einen off-grid oder noch nicht verfügbaren Schätzzeitpunkt.

6.7 KANALABHÄNGIGE AUSWAHL, DIAGNOSE UND PHASENBEISPIEL

SionnaMimoBeamformingConfiguration bündelt die explizite Methode MRT oder SVD, die unveränderten MimoSvdPrecodingOptions, MimoFeedbackConfiguration und MimoCsiTimingConfiguration. SionnaMimoPrecodingSelection besitzt den ausgewählten festen Precoder, CSI- und Sendezeitpunkt, CSI-Alter und, sofern vorhanden, die ursprünglichen SVD- und Feedback-/Alterungsergebnisse. Der Helper berechnet oder interpretiert diese Diagnosen nicht neu.

```
    configuration.mode = SionnaMimoMode::SU_BEAMFORMING;
    configuration.maxNss = 1;
    configuration.beamforming = SionnaMimoBeamformingConfiguration{};
    configuration.beamforming->method = SionnaMimoBeamformingMethod::SVD;
    configuration.beamforming->feedback.codebook = MimoFeedbackCodebook::SU_FINE;
    configuration.beamforming->csiTiming.updateInterval = MilliSeconds(5);
    configuration.beamforming->csiTiming.feedbackDelay = MilliSeconds(2);

    SionnaMimoHelper mimo(configuration);
    const auto& selection = mimo.SelectPrecoding(
    channelEstimateAtCsiTime,
    txNodeId, rxNodeId,
    csiTimestamp, transmissionTimestamp);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

SelectPrecoding validiert Konfiguration, CSI-Dimensionen und Zeitraster, erstellt das Ergebnis zunächst als lokalen Kandidaten und ersetzt den gespeicherten Zustand erst nach vollständigem Erfolg. SetConfiguration sowie Änderungen einer Knotenorientierung verwerfen die Auswahl. Apply lehnt SU_BEAMFORMING ohne vorherige Auswahl ab und installiert sonst ausschließlich selection.GetPrecodingMatrix() durch SetPrecodingMatrix. Der Adapter, das Signalmodell, die Leistungsnorm $\lVert W[k]\rVert_F^2=N_s$ und die 1x1-/Open-Loop-/Festprecodingpfade bleiben unverändert.

Das neue Programm ns3_wifi_sumimo_precoding.cc führt vier getrennte UDP-/SpectrumWifiPhy-CPU-Läufe aus. Open Loop und SVD verwenden denselben normalisierten 2x2-Kanal; fester Precoder und MRT verwenden denselben 2x1-MISO-Kanal, sodass der MRT-Gewinn direkt gegen die erste feste Antenne vergleichbar ist. Alle Fälle verwenden dieselbe Gesamtleistung P; die unterschiedlichen Frobeniusnormen $\sqrt{N_s}$ kompensiert das unveränderte Signalmodell durch P/N_s.

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --referenceSnrDb=16 --packetSize=1200 \
    --eesmBetaLinear=2"
```

Der Referenzlauf liefert für festes W[k] 16,000000 dB und 24,931200 Mbit/s, für MRT den analytischen Gewinn auf 19,010300 dB und 68,582400 Mbit/s und für Ein-Stream-SVD 21,563025 dB und 68,697600 Mbit/s. Die Precodingnorm bleibt in allen Ein-Stream-Fällen eins; Open Loop verwendet bei zwei Streams $\sqrt 2$. Das Beispiel bricht ab, falls die Gesamtleistungsidentität, die analytische MRT-SINR oder der ideale SVD-Gewinn verletzt wird.

Quantisierung und Alterung sind keine versteckten Defaults, sondern ausdrückliche Optionen:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --quantization=coarse \
    --csiUpdateIntervalMs=5 --feedbackDelayMs=2 \
    --transmissionTimeMs=13 --agingRotationDegrees=25"
```

quantization akzeptiert none, coarse und fine.
csiUpdateIntervalMs, feedbackDelayMs und transmissionTimeMs bestimmen

den nach Abschnitt 5.12 zulässigen CSI-Zeitstempel;
agingRotationDegrees verändert ausschließlich den aktuellen Sendekanal.
Wasserfüllung wird separat und weiterhin nur mit unquantisiertem aktuellem
CSI aktiviert:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --svdStreams=2 --waterFilling=1"
```

Jede Ausgabezeile besteht aus whitespace-getrennten schlüssel=wert-Feldern.
Die configuration-Zeile enthält Referenz-SNR, konstante Gesamtleistung,
Quantisierung, Wasserfüllschalter, Aktualisierungsintervall,
Feedbackverzögerung, CSI-Alter, kontrollierte Kanalrotation und das explizite
EESM-Beta. Jede scenario-Zeile enthält mindestens:

- throughput_mbps, expected_throughput_mbps und per;
- stream_sinr_db als kommaseparierte Streamliste sowie
effective_sinr_db;
- channel_rank, precoding_norm und power_fractions;
- csi_age_ns, wobei -1 einen nicht-CSI-basierten Modus bezeichnet;
- feedback_bits, chordal_error und rms_coefficient_error.

Eine abschließende validation=PASS-Zeile macht den Prozessstatus zusätzlich
maschinenlesbar. Die ExampleAsTest-Referenzdateien fixieren alle Felder für
den idealen, quantisiert-gealterten und Wasserfüllungsfall.

Die Migration ist additiv: Bestehende SISO-, Open-Loop- und
Festprecodingprogramme ändern weder Modus noch Aufrufreihenfolge. Nur
kanalabhängige Programme wählen SU_BEAMFORMING, füllen beamforming, rufen
nach Vorliegen einer expliziten CSI-Matrix einmal SelectPrecoding auf und
verwenden danach das bestehende ApplyForHeMcs. ClearSelectedPrecoding
ermöglicht eine bewusste Neuwahl. Es gibt weiterhin keinen automatischen
Simulator-Timer, keine implizite CSI-Abfrage und keinen MAC-Feedbacktransport;
diese Grenzen verhindern, dass das explizite Forschungsmodell eine
vollständige 802.11-Soundingprozedur vortäuscht.

7. VALIDIERUNG DER UNTERPHASEN 1.1 BIS 4.2

Die native Suite sionna enthält nun 31 schnelle Fälle:

1. API-Defaulttest für den vollständigen legacy-kompatiblen SISO-Vertrag,
   leere EESM-Kalibrierung, fehlendes W[k] und exakt drei gemeldete
   Betriebsarten.
2. Gültiger 2x2-Vertrag mit dual polarisiertem TX-Element, knotenspezifischer
   Orientierung, explizitem Beta, Identitäts-Precoding, frequenzselektiver
   2x1-Streamauswahl sowie zulässigem 2-TX-/1-RX-/1-Stream-Grenzfall.
3. Zentrale Negativmatrix für nicht endliche, unvollständige,
   widersprüchliche oder nicht unterstützte API-Konfigurationen einschließlich
   Precoding-TX-/NSS-Dimension, unzulässiger Beamforming-/Feedback-/Alterungs-
   Kombinationen, MU-MIMO und fehlendem MCS-Beta.
4. Manuell und über SionnaMimoHelper konfigurierte 1x1-Integration mit
   bytegleicher Array-/Orientierungs-IPC-Nachricht, identischer installierter
   SpectrumWifiPhy-Antennen-/NSS-Konfiguration, gleicher Rauschzahl und
   EESM-Kalibrierung sowie exakt gleichen MMSE-, EESM- und PER-Ergebnissen.
5. Derselbe Integrationsvergleich für eine vollständige offene
   2x2-Konfiguration ohne W[k].
6. Manuell gesetztes und durch SionnaMimoHelper angewendetes
   frequenzselektives komplexes 2x1-W[k] liefern dieselben Koeffizienten,
   installierten NSS, Post-MMSE-SINRs, EESM-SINR und PER.
7. Layouttest eines 2 x 2 x 2-CFR-Tensors mit eindeutig nummerierten
   Koeffizienten.
8. Precoding-Vertragstest für sc/tx/stream-Reihenfolge, explizite

   Frequenzflach-Expansion, Frobenius-Leistung sowie Dimensions-, Endlichkeits- und Normierungsfehler.
9. Reversible dB-/Linear-Umrechnung einschließlich 10 dB gleich Faktor zehn.
10. Link-Budget-Test für Sendeleistung, Pfadverlust, \(B\), \(T\) und \(NF\) sowie die erwartete Halbierung der SNR bei Verdopplung von \(B\) oder \(T\).
11. Flacher orthogonaler 2x2-Kanal: EESM erhält identische Tone-/Stream-SINRs unabhängig vom positiven Test-Beta.
12. Frequenzselektiver SISO-Kanal: EESM stimmt mit der analytischen Zweitonformel für das konfigurierte Beta überein.
13. SISO-Parität zur direkten ns-3.40-ErrorRateModel-Abfrage für HE MCS 0, 3, 7 und 11, neun SNR-Punkte von -5 bis 35 dB sowie 32, 400 und 1458 Byte. Das sind 108 Kombinationen; MCS 11 prüft auch den TableBased-Fallback auf Yans.
14. Monotone PER auf einem normierten frequenzselektiven Vier-Ton-Kanal für dieselben vier MCS und drei Paketgrößen bei 26 SNR-Punkten von -10 bis 40 dB. Die 312 geprüften Werte bilden je Konfiguration eine nicht steigende Folge mit einem echten PER-Abfall.
15. Exakte Reproduzierbarkeit von Post-MMSE-SINR, EESM-SINR und PER über 20 Wiederholungen eines komplexen frequenzselektiven 2x2-Kanals.
16. SISO-Test mit \(h=2+j\) und \(\rho=10\); erwartet wird \(\gamma=50\).
17. Orthogonaler 2x2-Kanal \(\mathbf H=\mathbf I\) und \(\rho=20\); jeder Stream erhält \(\gamma=10\) und \(\eta=2\log_2(11)\).
18. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei \(\rho=8\); erwartet werden (4,16), (16,4) und die analytische MI-Kompression.
19. Fester-Precoding-Referenztest für komplexes SISO, Identitätsparität, indexgenaues komplexes \(H[k]W[k]\), identische MMSE-Ausgabe zum vorberechneten effektiven Kanal und 2x2 mit einem ausgewählten Stream.
20. Ein-Stream-MRT/MRC-Test für frequenzselektives komplexes 2x1-MISO, Einheitsleistung von W[k], analytischen Arraygewinn \(\rho\lVert h\rVert_2^2\), gemeinsame Phasendrehung, äquivalentes 1x2-SIMO-MRC, exakte 1x1-Parität sowie die eindeutige Ablehnung von Nullkanal und allgemeinem 2x2-MIMO.
21. Deterministischer SVD-/Leistungsallokationstest für diagonale, unitär gedrehte, frequenzselektive, rangdefiziente und gleichsinguläre Kanäle. Er prüft explizite Stream-/Ranggrenzen, den relativen Rangschwellwert, Phasen-/Entartungskonvention, 1x1-Parität und analytische Eigenmode-SINRs. Zwei Zweimodenfälle unterhalb und oberhalb der Wasserfüllschwelle prüfen aktive Moden, endliche nichtnegative Anteile, exakte Leistungserhaltung, Wasserstand, KKT-Bedingungen und mindestens die Gleichleistungsrate.
22. Mindestkanalrang eins beziehungsweise zwei für Rang-1- und orthogonale 2x2-Kanäle über alle Subcarrier.
23. Spectrumadapter-Test für exaktes Rauschleistungs- und AWGN-Diversity-Compensation-Mapping sowie SISO-, Identitäts- und 2x1-Precoding ohne doppelte Anwendung von Rauschen, Leistung oder Arraygewinn.
24. Deserialisierung einer alten Nachricht ohne Dimensionsfelder als \(N_\mathrm{sc}\times1\times1\).
25. Protocol-Buffer-Rundlauf einer synthetischen 2 x 2 x 2-CFR mit den komplexen Werten 1-j bis 8-8j und indexgenauer Prüfung.

26. Helperauswahl für exaktes MRT, quantisierte gleichleistungs-SVD mit vollständiger Zustandsidentität und drei Millisekunden CSI-Alter sowie räumliche Wasserfüllung. Der Fall vergleicht jedes W[k] mit der bereits abgeschlossenen Fabrik, prüft Feedbackbits, Leistungsanteile und Gesamtleistung, weist nach, dass Apply exakt dieselben Koeffizienten am Adapter installiert, und verwirft die Auswahl nach einer Orientierungsänderung.
27. Protocol-Buffer-Rundlauf eines absichtlich umgekehrt eingegebenen Zweinutzer-Snapshots. Er prüft die sortierte Nutzerfolge 3, 9, gerichtete TX-/RX-Identität, gemeinsamen Zeitstempel und Frequenzraster, sc/rx/tx-Indizes sowie heterogene 1x1- und 1x2-RX-Arrays bei gemeinsamem

   Zweiantennensender.
28. Atomare Negativmatrix für doppelte Nutzer, gemischte Zeitstempel und
    Richtungen, abweichende Frequenzraster sowie eine RX-Geometrie, die nicht
    zur CFR-Dimension passt. Jeder Fall wird vor Cacheverwendung als
    `invalid_argument` abgelehnt.
29. Fester-Downlink-MU-Referenzfall mit expliziter Nutzer-/Streamzuordnung,
    exakter Gesamtleistung bei gleichen und ungleichen Spaltengewichten,
    Einzelnutzerparität zu Phase 3 einschließlich 1x1, null Interferenz für
    orthogonale Nutzer und analytischer gewünschter Leistung,
    Inter-User-Interferenz sowie SINR für korrelierte Kanäle.
30. Nutzerspezifischer HE-Vertrag mit zwei MCS, EESM-Beta, Link Budgets und
    Paketgrößen. Beide effektiven SINRs und Paketwahrscheinlichkeiten werden
    gegen direkte Aufrufe des bestehenden EESM- und ErrorRateModel-Vertrags
    geprüft.
31. Atomare Phase-4.2-Negativmatrix für doppelte oder lückenhafte
    Nutzer-/Streamzuordnung, fehlende oder unbekannte Nutzer, Uplinkrichtung,
    abweichendes Frequenzraster, fehlende oder nicht endliche SNR,
    unvollständige HE-Eingaben, NSS-Widerspruch und Paketgröße null. Ein danach
    ausgeführter gültiger Referenzfall weist den unveränderten Vertrag nach.

Die eigenständige native Suite `sionna-mimo-csi-feedback` ergänzt zwei Fälle
für Unterphase 3.4. Der Feedbackfall prüft HE und EHT, beide SU-Codebücher,
Bitbreiten und Payloadgrößen, exakt rekonstruierte Grenzindizes, deterministische
Wiederholung, den allgemeinen 3x2-Mehrstromfall, Leistungsrenormierung,
Phasen-/Subraumfehler sowie die Ablehnung nicht semi-unitärer Wasserfüllungs-
Precoder und ungültiger Indizes. Der Alterungsfall prüft Nullalter, getrennte
CSI- und Sende-CFR, Rasterung mit Aktualisierungsintervall und Rückmeldedauer,
statische und orthogonal gedrehte Kanäle, den daraus folgenden SINR-/PER-Abfall,
frische CSI, vollständige Zustandsidentität und den unveränderten 1x1-Fall.

Die sieben reinen Python-Vertragstests prüfen Standard- und explizite
Arraykonfiguration, bytegleiche 1x1-Serialisierung, die sc/rx/tx-Reihenfolge
einer synthetischen 2x2-CFR, die gemeinsame MIMO-Normalisierung sowie die
legacy-kompatible und explizite Knotenorientierung. Zwei Phase-4.1-Fälle
ergänzen denselben Zweinutzer-Rundlauf und die Negativmatrix für Identität,
Richtung, Zeitpunkt, Frequenzraster und heterogene RX-Geometrien. Diese
Tests benötigen weder Raytracing, ZMQ noch eine GPU. Die analytischen
Vergleiche des numerischen Kerns verwenden weiterhin eine absolute Toleranz
von 1e-10; dies gilt auch für Identitäts-Precoding und synthetische
\(H[k]W[k]\)-Referenzen sowie MRT-/MRC- und SVD-Eigenmode-SINRs.
Precoderleistung, Phaseninvarianz, effektiver MRT-Kanal, deterministische
SVD-Basis, Leistungsanteile, Wasserstand, KKT-Gleichheit und die neue
Feedback-Leistungsidentität werden innerhalb 1e-12 geprüft. Die
Link-Budget-/EESM-Vergleiche verwenden ebenfalls 1e-12.
Die Phase-4.2-Parität und Post-MMSE-SINR verwenden ebenfalls $1O^{-10}$;
gewünschte Leistung, Inter-User-Interferenz, Precoderleistung, EESM und
ErrorRateModel-Parität werden innerhalb $1O^{-12}$ geprüft.

Der reale Server-Integrationstest konfiguriert 1x2-PlanarArrays an TX und RX,
führt Sionna RT auf der CPU aus und prüft die Form 2 x 2 x 256, 1024 komplexe
Werte und mittlere normierte Leistung eins. Die übrigen sieben Servertests
decken den unveränderten 1x1-Pfad, Mobilität, Kohärenzzeit und Mehrnutzer-
Anfragen ab.

Die ExampleAsTest-Suites enthalten nun zwölf Integrationsprozesse. Die fünf
bestehenden Prozesse führen 1x1, Vollrang-2x2, Rang-1-2x2,
Identitäts-Precoding und 2x1-Streamauswahl über UDP/SpectrumWifiPhy aus und
prüfen widersprüchliches NSS, fehlendes MCS-Beta sowie den
CFR-/PHY-Dimensionsguard. Beamforming ist dabei nicht länger pauschal
reserviert.

Drei neue positive Prozesse führen das Phasenbeispiel mit idealer Auswahl, grob quantisiertem und drei Millisekunden altem CSI sowie räumlicher Wasserfüllung vollständig auf der CPU aus. Die Referenzlogs vergleichen alle maschinenlesbaren Felder. Vier neue negative Prozesse prüfen fehlende SelectPrecoding-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und einen nicht zum Aktualisierungsraster passenden CSI-Zeitstempel. Sie enden jeweils früh mit Status 2 und einer stabilen configuration_error-Zeile.

7.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 15. August 2026 wurden nach Abschluss der Unterphase 3.5 folgende Prüfungen auf einem CPU-System erfolgreich ausgeführt:

- gezielter Build von Bibliothek, nativer Testsuite, neuem Precoding-Vergleich und bestehendem SU-MIMO-Beispiel unter GCC 13.3
- native ns-3-Suite sionna: 26 von 26 Fällen bestanden. Der neue Helperfall vergleicht MRT und SVD einschließlich Feedback, Alterung und Wasserfüllung direkt mit den bereits vorhandenen wertbasierten Fabriken, prüft die installierten Koeffizienten und Leistungsanteile sowie die Invalidierung einer alten Auswahl. Der bestehende SVD-/Leistungsallokationsfall prüft zwei frequenzselektive diagonale Moden, unitäre Rotation, gleiche Singularwerte, 1x1-Parität, explizite Stream-/Rangfehler und nicht endliche Eingaben. Wasserfüllung aktiviert bei $\rho=0{,}5$ nur die starke Mode und bei $\rho=4$ beide Moden mit den analytischen Anteilen 0,59375 und 0,40625; Leistung, Wasserstand, KKT-Gleichheit und ideale Rate stimmen mit den Referenzen überein. Die bestehenden Phase-1-, Phase-2- und Phase-3.1-/3.2-Fälle bleiben bestanden
- native ns-3-Suite sionna-mimo-csi-feedback: zwei von zwei Fällen bestanden; deterministische HE-/EHT-Quantisierung und Rekonstruktion, Leistungs- und Fehlerdiagnostik, allgemeines 3x2-SU-Feedback sowie CSI-Alterung mit expliziter Zeit- und Zustandsidentität stimmen mit den analytischen Referenzen überein. Der Alterungstest weist zusätzlich den erwarteten SINR-/PER-Verlust bei orthogonal gedrehtem Sendekanal nach
- SISO-Paritätsmatrix: alle 108 Kombinationen stimmen innerhalb 1e-12 mit der direkt aufgerufenen ns-3-Fehlermodellschnittstelle überein
- frequenzselektive Monotonitätsmatrix: alle 312 PER-Werte bestanden
- Reproduzierbarkeit: 20 identische 2x2-Aufrufe mit exakt gleichen Ergebnissen
- ExampleAsTest sionna-example-sumimo-minimal: ein von einem Fall bestanden; Vollrang-2x2 erreicht 134,3904 statt 68,1408 Mbit/s bei 1x1, Rang-1-2x2 erreicht bei PER 1 erwartungsgemäß 0 Mbit/s; Identitäts-Precoding reproduziert die analytischen Vollrangwerte und die 2x1-Auswahl erreicht 23,010300 dB bei voller Gesamtleistung
- ExampleAsTest sionna-example-sumimo-precoding: drei von drei Prozessen bestanden; idealer Vergleich, grob quantisiertes und drei Millisekunden altes CSI sowie Wasserfüllung liefern die vollständigen stabilen Diagnostikzeilen und validation=PASS
- ExampleAsTest sionna-example-sumimo-precoding-negative: vier von vier Prozessen bestanden; fehlende Precoder-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und ein off-grid CSI-Zeitstempel liefern jeweils die erwartete configuration_error-Zeile
- ExampleAsTest sionna-example-mimo-helper-configuration-negative: drei von drei Fällen bestanden; ungültiges NSS, nicht unterstütztes MU-MIMO und fehlendes MCS-Beta brechen jeweils früh mit Status 2 und eindeutiger Meldung ab
- ExampleAsTest sionna-example-mimo-helper-runtime-negative: ein von einem Fall bestanden; eine helperkonfigurierte 2x2-PHY mit 1x1-CFR erreicht den erwarteten Dimensionsguard im echten SpectrumWifiPhy-Laufzeitpfad
- reine Python-Vertragssuite: fünf von fünf Fällen in 0,001 s bestanden; der separate Legacy-Importlauf bestätigte Sionna 1.2.1, Mitsuba 3.7.1, NumPy 1.26.4, TensorFlow im CPU-Modus ohne GPU und ZeroMQ

  - gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 153,015 s bestanden;
  der reale 2x2-Fall lief in 1,6164 s, lieferte 2 x 2 x 256 und erfüllte die
  Normalisierung innerhalb 1e-6
  - P2P-End-to-End-Test example-sionna-sensing im unveränderten 1x1-Modus:
  drei 1 x 1 x 3072-CFR-Antworten, zwei empfangene UDP-Pakete zu je 1024 Byte
  und Cache-Hit-Rate 0,993056

  7.2 MIGRATION UND VALIDIERUNG MIT SIONNA 2.0.1

Am 20. August 2026 wurde die Python-Serverumgebung auf Sionna 2.0.1 und
Sionna RT 2.0.1 aktualisiert. Die reproduzierbare Abhängigkeitsbasis lautet:

  - `requirements.txt` fixiert Sionna 2.0.1, Sionna RT 2.0.1,
  Mitsuba 3.8.0, Dr.Jit 1.3.1 und PyTorch 2.9.1; die von Sionna geforderte
  NumPy-Untergrenze ist 2.2.6.
  - `requirements_cpuonly.txt` wählt zusätzlich das offizielle CPU-Rad
  PyTorch 2.9.1+cpu.
  - TensorFlow wurde aus beiden Requirements und aus dem Server entfernt.
  Der Server erkennt den tatsächlich von Sionna RT gewählten
  Mitsuba-Variantennamen und unterscheidet darüber CUDA- und LLVM-Ausführung.
  - Der ns-3-Seed wird bei beiden Raytracing-Pfaden nun mit `seed` explizit
  an `PathSolver` weitergegeben. Damit steuert der Simulationsseed auch die
  stochastische Strahlabtastung; zuvor blieb dort implizit Sionnas
  Standardwert aktiv.
  - `Paths.cir()`, `Paths.cfr()`, `PlanarArray` und `PathSolver` behalten
  in Sionna RT 2.0.1 die hier verwendete API. Die CFR-Dimension
  [num_rx, num_rx_ant, num_tx, num_tx_ant, num_time_steps, num_subcarriers]
  und damit der bestehende sc/rx/tx-Datenvertrag bleiben unverändert.

Die aktualisierte CPU-Umgebung bestand folgende Prüfungen:

  - strikte Importprüfung für Sionna 2.0.1 und Sionna RT 2.0.1 sowie
  Mitsuba 3.8.0, Dr.Jit 1.3.1, NumPy 2.5.2, PyTorch 2.9.1+cpu und ZeroMQ
  - Python-Vertragssuite: fünf von fünf Fällen in 0,003 s bestanden
  - gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 155,993 s bestanden;
  der reale MIMO-Fall lieferte auf dem LLVM-Backend erneut eine endliche,
  gemeinsam über alle Antennenpaare normalisierte 2 x 2 x 256-CFR
  - native ns-3-Suite `sionna`: 26 von 26 Fällen bestanden
  - native ns-3-Suite `sionna-mimo-csi-feedback`: zwei von zwei Fällen bestanden

Damit ist die Sionna-RT-Schnittstelle für 1x1 und 2x2 unter 2.0.1 validiert.
Der Wechsel von TensorFlow zu PyTorch betrifft den vorhandenen RT-Kanalpfad
nicht; TensorFlow ist für ns3sionna nach der Migration keine
Runtimeabhängigkeit mehr.

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt:
Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen
Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet
ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon
nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

  7.3 VALIDIERUNG DER UNTERPHASE 4.1

Am 20. August 2026 wurden nach der Implementierung von Unterphase 4.1
folgende gezielte Prüfungen erfolgreich ausgeführt:

  - Build des Ziels `sionna-test` einschließlich neu erzeugter
  Protocol-Buffer-Bindings
  - native ns-3-Suite `sionna` mit allen 28 registrierten Fällen; die zwei neuen
  Fälle decken Rundlauf, Sortierung, Identität, Richtung, heterogene
  RX-Dimensionen und die atomare Negativmatrix ab

- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,004 s
- gesamte Sionna-CPU-Serversuite mit acht von acht Fällen in 171,865 s, einschließlich der vorhandenen Mehrnutzer-, 1x1- und 2x2-Pfade
- unveränderte SU-Suites `sionna-mimo-csi-feedback`, `sionna-example-sumimo-minimal` und `sionna-example-sumimo-precoding`; alle Fälle bestanden
- `sionna-example-mimo-helper-configuration-negative` mit allen drei Fällen; insbesondere bleibt der MU-MIMO-Modus des Helpers abgelehnt und nimmt damit Unterphase 4.2 nicht vorweg

7.4 VALIDIERUNG DER UNTERPHASE 4.2

Am 21. August 2026 wurden nach der Implementierung von Unterphase 4.2 folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test` mit den vier neuen nativen Quelldateien
- native ns-3-Suite `sionna` mit allen 31 registrierten Fällen; die drei neuen Fälle decken den festen MU-Precoder, Einzelnutzer- und 1x1-Parität, orthogonale und korrelierte Referenzkanäle, Gesamtleistung, nutzerweise EESM/PER und die Negativmatrix ab
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,005 s; damit bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`, `sionna-example-sumimo-precoding` mit allen drei positiven Prozessen, `sionna-example-sumimo-precoding-negative`, `sionna-example-mimo-helper-configuration-negative` und `sionna-example-mimo-helper-runtime-negative` erfolgreich
- `git diff --check` ohne Befund

Die Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt: Unterphase 4.2 ändert weder Python noch Protocol Buffers, Serialisierung, Batching, Cache, ZMQ oder Sionna RT. Die schnelle Python-Vertragssuite und die nativen Phase-4.1-Fälle prüfen die unveränderten Grenzen direkt.

8. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- SionnaMimoHelper überträgt eine gemeinsame TX- und RX-Geometrie für die jeweilige integrierte Verbindung. Knoten können unterschiedliche Orientierungen besitzen. Der Snapshotvertrag aus Unterphase 4.1 repräsentiert heterogene RX-Geometrien pro Nutzer; die aktuelle Sionna-Raytracing-Konfiguration verwendet weiterhin ein gemeinsames RX-Array.
- Nicht explizit orientierte Knoten behalten die alte Serverorientierung [0, -180, 0]. Explizite Winkel werden in beiden Servermodi an Sionna übertragen.
- Der integrierte MIMO-Pfad unterstützt HE SU ohne Precoding, festes benutzergegebenes $W[k]$ und die explizite Helperwahl von MRT oder SVD. Nicht-HE-Kontrollrahmen verwenden weiterhin das erste Antennenpaar. SelectPrecoding benötigt eine bereits vorliegende CSI-Matrix und Zeitstempel; automatische Kanalschätzung, Sounding, periodische Simulatorereignisse, MAC-Feedbackframes und per-Paket-Aktualisierung sind nicht implementiert. Unterphase 4.2 stellt den festen gemeinsamen MU-Precoder und die Inter-User-Interferenz als direkte native API bereit. Installation durch SionnaMimoHelper, HE-MU-PHY-Metadaten, Scheduler und automatische Precoderwahl beginnen frühestens mit Unterphase 4.3 beziehungsweise 4.4.
- MRT bleibt der eindeutige perfekte 2x1-MISO-Ein-Stream-Fall. Quantisierung und positives CSI-Alter gelten nur für gleichleistungs-SVD. Wasserfüllung bleibt auf unquantisiertes aktuelles CSI beschränkt, weil die semantische Winkelrückmeldung einen semi-unitären Gleichleistungs-Precoder verlangt.

- Die Auswahl speichert einen einzelnen unveränderlichen W[k]-Schnappschuss pro Helper. Der Anwender muss die aktuelle Sendekanal-Matrix weiterhin getrennt für SINR und PER bereitstellen und bei neuer CSI SelectPrecoding erneut aufrufen. Das ist kein per-Link- oder per-Paket-Precodercache.
- Die Inter-User-Interferenz der gleichzeitig durch denselben festen MU-Precoder gesendeten Streams geht in Unterphase 4.2 frequenz- und antennenabhängig in die MMSE-Kovarianz ein. Andere zeitvariable externe Interferenz wird weiterhin erst nach der MMSE-/EESM-Abstraktion durch InterferenceHelper skalar berücksichtigt; ihre räumliche Kovarianz ist nicht modelliert.
- SionnaMimoHelper übernimmt die Rauschzahl einmalig und setzt sie identisch am Adapter sowie an beiden PHY-Helpern. Code ohne High-Level-API muss beide Werte weiterhin manuell konsistent halten, da ns-3.40 das PHY-Attribut nicht lesbar macht.
- Es existiert absichtlich keine mitgelieferte HE-EESM-Beta-Tabelle. Frequenzselektive absolute PER-Prognosen benötigen vor der wissenschaftlichen Nutzung eine dokumentierte Link-Level-Kalibrierung. Ohne diese Konfiguration bricht das Modell früh ab.
- TableBasedErrorRateModel ist eine AWGN-Referenz. Seine SNR-/PER-Kurven und Paketlängenextrapolation stammen aus ns-3; ihre Grenzen gelten unverändert.
- Perfekte RX-CSI bleibt bewusst idealisiert. Für die Precoderwahl trennt der neue Alterungspfad die geschätzte CFR zum CSI-Zeitpunkt explizit von der CFR zum Sendezeitpunkt und weist Alter, Aktualisierungsintervall und Rückmeldedauer aus; eine automatische Kanalschätzung oder Ablaufsteuerung im Helper existiert noch nicht.
- Ohne W[k] gilt weiterhin genau ein Stream je TX-Antenne. Mit festem W[k] sind kontrollierte Streamauswahl und $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$ für SU implementiert. Im festen MU-Kern gilt zusätzlich $N_{s,\mathrm{ges}}\leq N_\mathrm{tx}$ und $N_{s,u}\leq N_{\mathrm{rx},u}$.
- Die MI-Kompression bleibt eine Diagnosemetrik; nur die explizit konfigurierte EESM-Ausgabe wird an ErrorRateModel übergeben.
- Explizit identifizierte Phase-4.1-Rohkanaleinträge werden nur in ihrer gespeicherten TX-/RX-Richtung getroffen. Nur identitätsfreie Legacy-Einträge behalten die bisherige reziproke Cacheverwendung. Die Precoder-Zustandsidentität enthält ebenfalls gerichtete TX-/RX-Knoten, beide Arraykonfigurationen, Modus, Feedbackkonfiguration und CSI-Zeitpunkt. Der Helper gibt diese Identität maschinenlesbar weiter, bindet sie aber bewusst nicht an einen automatischen Laufzeitcache.
- Der deterministische Beispielkanal ist eine gerichtete, normalisierte Vorwärts-CFR. Das Ad-hoc-UDP-Beispiel setzt deshalb den ARP-Eintrag am Sender statisch; so erzeugt die Infrastruktur keine rückwärts gerichtete HE-Datennachricht mit inkompatiblen 2x1-Dimensionen. Normale Sionna-Cachepfade sind davon nicht betroffen.

Phase 3 ist mit Unterphase 3.5 abgeschlossen; Unterphase 4.1 ergänzt den gerichteten, atomar validierten Mehrnutzer-Kanalsnapshot. Unterphase 4.2 verwendet ihn nun für einen festen, explizit zugeordneten gemeinsamen Downlink-Precoder, erhält die gesamte AP-Leistung und liefert gewünschte Leistung, Inter-User-Interferenz, Post-MMSE-SINR, EESM und PER je Nutzer. Der Helper führt parallel die unveränderten SU-Verträge durch den bisherigen festen-W[k]-Pfad. Einzelnutzer- und 1x1-Grenzfall sowie alle Phase-1-/2-/3-Pfade bleiben kompatibel. Automatische Sounding-/Feedbackabläufe, Precoderwahl einschließlich ZF/RZF, ein MU-Precoder-Laufzeitcache, Scheduler, HE-MU-PHY-Integration und sämtliche Unterphasen ab 4.3 wurden nicht begonnen.

9. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
./ns3 configure --enable-examples --enable-tests
```

```
    ./ns3 build
    ./test.py -s sionna
    ./test.py -s sionna-mimo-csi-feedback
    ./test.py -s sionna-example-sumimo-minimal
    ./test.py -s sionna-example-mimo-helper-configuration-negative
    ./test.py -s sionna-example-mimo-helper-runtime-negative
    ./test.py -s sionna-example-sumimo-precoding
    ./test.py -s sionna-example-sumimo-precoding-negative
    ./ns3 run "ns3_wifi_sumimo_minimal --eesmBetaLinear=2"
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2"
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2 \
    --quantization=coarse --csiUpdateIntervalMs=5 --feedbackDelayMs=2"

Python-Vertrag und Server vom Verzeichnis contrib/sionna/model/ns3sionna:

    python -m pip install --upgrade -r requirements_cpuonly.txt
    python tests/test_imports.py
    conda run -n sionna-venv python -m unittest -v tests.test_mimo_data_contract
    conda run -n sionna-venv python -m unittest -v tests.unit_tests

PDF neu erzeugen:

    conda run -n sionna-venv python
    contrib/sionna/tools/render_model_pdf.py
    contrib/sionna/ns3sionna_mimo_model.md
    contrib/sionna/ns3sionna_mimo_model.pdf

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.
```